Conventional ML Essentials

A Comprehensive Interview Reference

Volume IV of the ML Interview Program

Interview Preparation Guide

September 4, 2026

Contents

Part I

Models

Chapter 1

Supervised Learning Core

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).

1.1 Linear Regression

1.2 Regularization: L1, L2, Elastic Net

1.3 Logistic Regression and GLMs

1.4 SVMs and Kernels

1.5 Naive Bayes

1.6 k-Nearest Neighbors

1.7 Interview Questions

Chapter 2

Trees and Ensembles

TL;DR

Gradient-boosted decision trees (GBDTs) are the single most-tested classical-ML topic in 2026 interviews, and for good reason: they remain the production default for tabular prediction at almost every platform company, and the XGBoost second-order derivation is the canonical "can you do math about a tool you use daily" probe. This chapter builds the full stack: CART mechanics (impurity, greedy split finding, pruning, why single trees are high-variance); bagging and random forests with the correlated-variance algebra that motivates feature subsampling; gradient boosting derived properly as functional gradient descent (residual fitting is the squared-loss special case, not the definition); the XGBoost objective with the closed-form leaf weight $w^* = -G/(H+\lambda)$ and split-gain formula derived end to end; LightGBM's histogram/leaf-wise/GOSS/EFB engineering; CatBoost's ordered target statistics and ordered boosting; the eight hyperparameters that actually matter; the honest 2026 state of GBDT vs. neural networks on tabular data; and TreeSHAP, monotonic constraints, and serving-latency practicalities. This is the program's canonical GBDT home—the learning-to-rank chapter [SR 5] builds LambdaMART on top of exactly this machinery.

2.1 Decision Trees: CART Mechanics

Everything in this chapter is built from one primitive: a binary decision tree grown by recursive greedy splitting, in the CART (Classification and Regression Trees, Breiman et al., 1984) tradition. Interviewers rarely ask about single trees for their own sake anymore—they ask because the tree-level details (impurity concavity, split-finding complexity, categorical handling, the variance problem) are exactly the load-bearing parts of every ensemble question that follows.

2.1.1 Greedy Recursive Partitioning

A CART tree partitions the feature space into axis-aligned boxes and predicts a constant in each box. Training proceeds top-down: at each node, consider every feature j and every threshold t, evaluate the candidate split $\{x_j \le t\}$ vs. $\{x_j > t\}$ by how much it improves a purity criterion, take the best, and recurse on the two children until a stopping rule fires.

Two facts frame everything else:

- **Greedy is a heuristic, not an optimum.** Finding the globally optimal decision tree is NP-hard (Hyafil & Rivest, 1976), so every practical learner is greedy and one-step myopic: a split that looks mediocre now but enables a great split below is invisible to it. Nobody fixes this—ensembles paper over it instead.
- **The prediction is piecewise constant.** A tree can only output values seen in its leaves (class votes, or means of training targets). This single property explains both the axis-aligned inductive bias that makes trees great on tabular data (Section 2.8) and their total inability to extrapolate a trend (Chapter 7).

2.1.2 Impurity Measures

For a classification node with class proportions $p_1, \dots, p_K$:

The Three Classification Impurities

$$\text{Gini:} \quad G = \sum_{k=1}^{K} p_k(1 - p_k) = 1 - \sum_{k=1}^{K} p_k^2 \tag{2.1}$$

$$\text{Entropy:} \quad H = -\sum_{k=1}^{K} p_k \log_2 p_k \tag{2.2}$$

$$\text{Misclassification error:} \quad E = 1 - \max_k p_k \tag{2.3}$$

All three are maximized at the uniform distribution and zero for a pure node. A split of node P (with n samples) into children L, R (with n_L, n_R) is scored by the **impurity decrease** ("information gain" when the impurity is entropy):

$$\Delta = I(P) - \frac{n_L}{n} I(L) - \frac{n_R}{n} I(R) \tag{2.4}$$

which is always ≥ 0 for concave I (Jensen's inequality).

Why Gini and entropy, not misclassification error? Because Gini and entropy are *strictly* concave in the class proportions, they reward a split that makes children purer even when no child's majority class changes—misclassification error is piecewise linear and often cannot tell two candidate splits apart. The standard worked example: a parent with $(4+, 4-)$.

- **Split A** produces children $(3+, 1-)$ and $(1+, 3-)$. Misclassification: each child errs at $1/4$, weighted error $= 1/4$. Gini: each child has $2 \cdot \frac{3}{4} \cdot \frac{1}{4} = 0.375$, weighted $= 0.375$.
- **Split B** produces children $(2+, 4-)$ and $(2+, 0-)$. Misclassification: $\frac{6}{8} \cdot \frac{1}{3} + \frac{2}{8} \cdot 0 = 1/4$—*identical* to Split A. Gini: $\frac{6}{8} \cdot \frac{4}{9} + \frac{2}{8} \cdot 0 = 0.333$—strictly better.

Split B has created a pure node, which is obviously the more promising tree to keep growing, and only the strictly concave criteria can see it. In practice Gini vs. entropy is a non-event: they disagree on roughly 2% of splits and yield indistinguishable trees; Gini is the common default because it avoids the log. Misclassification error still has a role in *pruning*, where the quantity you actually care about is held-out error, not growth potential.

A worked information-gain computation (interviewers ask for numbers, not formulas). Eight emails, 4 spam / 4 ham, so parent entropy $H = 1$ bit. Candidate feature `contains_link`: splits into (3 spam, 1 ham) and (1 spam, 3 ham); each child has $H = -\frac{3}{4}\log_2\frac{3}{4} - \frac{1}{4}\log_2\frac{1}{4} =$

0.811, weighted 0.811, so IG $= 1 - 0.811 = 0.189$ bits. Candidate feature `all_caps_subject`: splits into (2 spam, 0 ham) and (2 spam, 4 ham); children have $H = 0$ and $H = -\frac{1}{3}\log_2\frac{1}{3} - \frac{2}{3}\log_2\frac{2}{3} = 0.918$, weighted $\frac{2}{8}\cdot 0 + \frac{6}{8}\cdot 0.918 = 0.689$, so IG $= 0.311$ bits. The second split wins despite touching fewer rows on its pure side—purity gain, weighted by mass, is the whole criterion, and being able to produce this arithmetic unprompted in under a minute is the difference between knowing the formula and owning it.

Regression trees replace impurity with the sum of squared errors around the leaf mean: splitting minimizes $\sum_{i\in L}(y_i - \bar{y}_L)^2 + \sum_{i\in R}(y_i - \bar{y}_R)^2$, i.e., maximizes **variance reduction**, and the leaf predicts $\bar{y}$ (the SSE-minimizing constant). Everything below about split finding carries over with class counts replaced by running sums.

2.1.3 Split Finding and Its Complexity

The naive algorithm—for each candidate threshold, re-partition the node and recount—is $O(n^2)$ per feature and is the classic thing interviewers watch you avoid in a coding round (Chapter 8). The right algorithm:

1. **Sort** the node's samples by feature j: $O(n \log n)$.
2. **Sweep** left to right, moving one sample at a time from the right child to the left child, updating class counts (classification) or the running $\sum y$ and $\sum y^2$ (regression) in $O(1)$ per step, evaluating the criterion at each boundary between distinct values (thresholds are conventionally midpoints between consecutive distinct values).

For regression the $O(1)$ update works because SSE $= \sum y_i^2 - (\sum y_i)^2/n$: two accumulators per side suffice. (This identity is also why an MAE splitting criterion is dramatically slower—there is no constant-time update for a median.) Per node this is $O(d\, n \log n)$; production implementations sort each feature *once* for the whole tree and maintain sorted order through splits, so a depth-D tree costs $O(d\, n \log n)$ for the presort plus $O(d\, n)$ per level, $O(d\, n(\log n + D))$ total. XGBoost's "exact greedy" mode is precisely this, engineered around a pre-sorted columnar layout (Section 2.4); LightGBM replaces the sorted sweep with histograms (Section 2.5).

2.1.4 Stopping and Cost-Complexity Pruning

Unregularized, a tree grows until every leaf is pure—memorizing the training set. Two control strategies:

Pre-stopping (what GBDT libraries use): `max_depth`, minimum samples (or hessian mass) per leaf, minimum impurity decrease. Cheap, but greedy stopping can quit right before a good split.

Post-pruning (classic CART): grow a large tree T_0, then minimize the **cost-complexity criterion**

$$R_\alpha(T) = R(T) + \alpha\,|T|, \tag{2.5}$$

where $R(T)$ is training error and $|T|$ the number of leaves. As α increases from 0, the optimal subtree shrinks through a *nested* sequence found by weakest-link pruning (repeatedly collapse the internal node whose removal costs the least per leaf saved); α is then chosen by cross-validation (sklearn's `ccp_alpha`). The idea survives in modern form: XGBoost's γ is exactly a per-leaf complexity charge applied *inside* the split-gain formula (Section 2.4), so pruning happens against the boosting objective rather than as a separate pass.

2.1.5 Categorical Features and Missing Values at the Tree Level

Categoricals. A categorical feature with k levels admits $2^{k-1} - 1$ possible binary partitions—exponential. Two classical escapes:

- **The sorting trick.** For binary classification or squared-error regression, sorting the categories by their mean target value and then scanning that ordering as if it were numeric provably finds the *optimal* binary partition (Fisher, 1958; used by CART and, in gradient-statistics form, by LightGBM). This turns $2^{k-1} - 1$ candidates into $k - 1$.
- **Encode first.** One-hot works for small k but explodes memory and produces many near-useless binary features at high cardinality (each split isolates one level); target encoding is powerful but is a target-leakage machine unless done out-of-fold—CatBoost's ordered target statistics (Section 2.6) are the principled version, and the general leakage taxonomy lives in Chapter 5.

Missing values. Classic CART used *surrogate splits*: at each node, learn backup splits on other features that best mimic the primary split, and route missing values by the best surrogate. Modern GBDTs do something simpler and better: learn a **default direction** per split by trying missing-goes-left and missing-goes-right and keeping whichever reduces the loss more (Section 2.4). The practical consequence—trees handle missing values natively and can even *exploit informative missingness*—is a genuine advantage over most NN pipelines, which must impute.

2.1.6 Why Single Trees Are Not Enough

The Tree Trade: Low Bias Available, High Variance Guaranteed

A deep tree is a low-bias, extremely high-variance estimator: it can represent nearly any function on the training data, but tiny perturbations of the data change the root split, and every decision below it, giving a completely different tree. Additionally, the axis-aligned, piecewise-constant hypothesis class means a diagonal linear boundary costs a staircase of many splits, and any smooth trend is approximated by steps that end at the training range. Ensembles exist to fix the variance problem while keeping the (useful) axis-aligned bias: **bagging/random forests average many high-variance trees to cancel variance; boosting adds many high-bias shallow trees to cancel bias.** That one sentence organizes the entire chapter.

Additional single-tree properties worth stating on demand: invariance to any monotone transform of a feature (only the sort order matters—so no scaling, no log-transforms needed for fit, though they may help interpretation); native mixed-type feature handling; interpretability of a *small* tree (a depth-3 tree is a flowchart; a 500-tree ensemble is not, hence Section 2.9); and the extrapolation ceiling (predictions are bounded by the range of training-leaf values—the standard time-series trap, Chapter 7).

2.2 Bagging and Random Forests

2.2.1 Bootstrap Aggregation and the Variance Algebra

Bagging (Breiman, 1996): draw B bootstrap samples (sample n points with replacement), train one deep tree on each, and average their predictions (majority vote for classification). Why this

works is a two-line variance calculation that interviewers ask verbatim.

Variance of an Average of Correlated Estimators

Let $T_1, \dots, T_B$ be identically distributed predictors with $\mathrm{Var}(T_b) = \sigma^2$ and pairwise correlation $\rho \geq 0$. Then

$$\mathrm{Var}\left(\frac{1}{B}\sum_{b=1}^{B} T_b\right) = \frac{1}{B^2}\Big[B\sigma^2 + B(B-1)\rho\sigma^2\Big] = \underbrace{\rho\sigma^2}_{\text{correlation floor}} + \underbrace{\frac{1-\rho}{B}\sigma^2}_{\to 0 \text{ as } B\to\infty} \tag{2.6}$$

Averaging kills the $\frac{1-\rho}{B}\sigma^2$ term but is powerless against $\rho\sigma^2$. Bagging leaves the (small) bias of a deep tree essentially unchanged—the average of identically distributed estimators has the same expectation—so **bagging is purely a variance-reduction device, and its effectiveness is capped by tree correlation.**

Bagged trees are substantially correlated in practice: every bootstrap sample contains most of the same strong features, so most trees pick the same root split and resemble each other. That residual $\rho\sigma^2$ is exactly the term random forests attack.

2.2.2 Random Forests: Decorrelation by Feature Subsampling

A random forest (Breiman, 2001) is bagging plus one modification: at *each split*, only a random subset of `max_features` ("mtry") features is eligible—classically $\sqrt{d}$ for classification, $d/3$ for regression. A dominant feature now sits out many splits, forcing trees to discover different structure. In the algebra above, feature subsampling lowers ρ at the cost of somewhat increasing each tree's σ^2 (and bias); the product $\rho\sigma^2$ usually drops sharply, which is the entire trick. Stated as a slogan for interviews: **bagging reduces variance; the random-subspace trick reduces the correlation that limits how much variance bagging can remove.**

Because variance decreases monotonically in B and bias is fixed, **adding trees to a random forest cannot overfit**—performance asymptotes, and the only cost of large B is compute. ("More trees overfit" is a red-flag confusion with boosting *rounds*, which absolutely do overfit.) The bias claim deserves its one-line justification: each tree is identically distributed, so $\mathbb{E}[\frac{1}{B}\sum_b T_b] = \mathbb{E}[T_1]$—averaging changes nothing about where the estimator points, only how much it wobbles. This is also why RF wants *deep, unpruned* trees: the base learner should carry as little bias as possible, since bias is the one thing the ensemble cannot remove.

Two variations worth knowing at name-drop depth. **Extremely randomized trees** (ExtraTrees; Geurts et al., 2006) push decorrelation further: thresholds are drawn at *random* per candidate feature rather than optimized, buying lower ρ (and faster training—no split sweep) at the price of more per-tree bias; on noisy data they sometimes edge out RF. And **subsampling without replacement** (∼63% draws, "subagging") behaves nearly identically to the bootstrap in practice—the bootstrap is not sacred, the perturb-and-average principle is.

2.2.3 Out-of-Bag Estimation

Each Tree Sees About 63.2% of the Data

The probability that a given training point is absent from one bootstrap draw is $(1 - \frac{1}{n})$; absent from all n draws,

$$\left(1 - \frac{1}{n}\right)^n \xrightarrow{n\to\infty} e^{-1} \approx 0.368. \tag{2.7}$$

So each tree trains on $\approx 63.2\%$ of the unique points; the remaining $\approx 36.8\%$ are that tree's **out-of-bag (OOB)** set. Predicting each point using only the trees that did *not* see it (about B/e of them) yields the OOB error—an approximately unbiased generalization estimate obtained **for free**, with no held-out split.

OOB error is the random forest's built-in cross-validation: use it to pick `max_features` and to monitor whether the forest has enough trees (OOB error plateaus). It is slightly pessimistic (each OOB prediction ensembles only $\sim B/e$ trees rather than B), which is worth saying before an interviewer does. OOB samples also power OOB permutation importance.

2.2.4 Hyperparameters, Honestly

The honest headline—backed by the tunability literature (Probst et al., 2019)—is that **random forests barely need tuning**: defaults (deep unpruned trees, $\sqrt{d}$ features, a few hundred trees) are near-optimal on most problems, which is precisely why RF remains the best "strong baseline before lunch" model. The knobs, in order of the little tunability that exists: `max_features` (the bias-correlation dial; the one worth a small sweep), `min_samples_leaf` (smooths predictions, helps on noisy targets), `n_estimators` (more is monotonically better; pick by compute budget and OOB plateau), `max_depth` (rarely needed; RF wants deep trees). Claiming a large RF tuning campaign was decisive is itself a mild red flag—the headroom is usually a point or two, unlike GBDTs where tuning genuinely matters (Section 2.7).

2.2.5 Feature Importance: MDI, Permutation, and the Traps

Mean decrease in impurity (MDI)—sum each feature's impurity decreases, weighted by node size, across the forest—is the "free" importance every library prints, and it is the one you should trust least:

- **Cardinality bias.** High-cardinality categoricals and continuous features offer far more candidate split points, so they win splits by chance and accumulate importance even when pure noise. A random ID column can outrank genuinely predictive features.
- **Computed on training data.** MDI credits splits that memorized noise; an overfit forest reports its overfitting as "importance."
- **Correlated features split credit arbitrarily**, depending on which one each tree happened to pick first.

Permutation importance—shuffle one feature column on *held-out* data and measure the metric drop—fixes the first two problems (it is model-agnostic and evaluated out-of-sample) but not the third: correlated features still dilute each other's scores, and permuting one member of a correlated pair creates off-manifold feature combinations the model never saw. The fuller trap catalog (grouped permutation, drop-column retraining, SHAP's versions of the same diseases) lives in Chapter 5; the interview-safe summary is:

Warning

Feature importance is a statement about *this model's internals*, not about the world. Never read MDI as ground truth (cardinality bias, train-set computation), never read any importance as causal, and treat a single dominant feature as a **leakage alarm** before treating it as an insight.

2.3 Gradient Boosting as Functional Gradient Descent

2.3.1 Forward Stagewise Additive Modeling

Boosting builds an additive model

$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta\, f_m(x), \tag{2.8}$$

where each f_m is a small tree (the "weak learner"), η is the learning rate, and F_0 is a constant base score (the target mean for squared loss; the log-odds prior for log loss). Training is **forward stagewise**: at stage m, all previous trees are frozen and we greedily pick the next tree to most reduce the loss:

$$f_m = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} L\big(y_i,\ F_{m-1}(x_i) + f(x_i)\big). \tag{2.9}$$

This inner problem is itself intractable over the space of trees $\mathcal{F}$, and the entire content of "gradient" boosting is the trick that makes it tractable.

2.3.2 The Functional Gradient Descent View

Gradient Boosting = Gradient Descent in Function Space

Treat the vector of training predictions $\boldsymbol{F} = (F(x_1), \ldots, F(x_n))$ as the "parameters." The steepest-descent direction for the empirical loss is the negative gradient with one coordinate per training point:

$$r_i^{(m)} = -\left.\frac{\partial L\big(y_i, F(x_i)\big)}{\partial F(x_i)}\right|_{F=F_{m-1}} \qquad \text{(the \textbf{pseudo-residuals}).} \tag{2.10}$$

An unconstrained gradient step $F(x_i)$ += $\eta\, r_i$ would only update predictions at the n training points—useless for generalization. Instead, **project the step onto the weak-learner class**: fit a small tree to the pseudo-residuals by least squares,

$$f_m = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{n} \big(r_i^{(m)} - f(x_i)\big)^2, \tag{2.11}$$

then update $F_m = F_{m-1} + \eta f_m$ (Friedman's TreeBoost additionally re-solves the optimal constant per leaf by line search). Each boosting round is one gradient-descent step in function space, with the tree as a smoothed, generalizable stand-in for the gradient; η is the step size.

Special cases every candidate must produce on demand:

- Squared loss $L = \frac{1}{2}(y - F)^2$: $r_i = y_i - F_{m-1}(x_i)$ — pseudo-residuals are *literal residuals*. This is where the folk description "boosting fits the residuals" comes from.
- Log loss with F a logit, $p_i = \sigma(F_{m-1}(x_i))$: $r_i = y_i - p_i$.
- Absolute loss: $r_i = \text{sign}(y_i - F_{m-1}(x_i))$ — fitting signs, not residuals, which is why quantile/MAE boosting behaves differently.

Warning

"Gradient boosting fits the residuals" stated as the general mechanism is a classic red flag. Residual fitting is the **squared-loss special case**; the general mechanism is fitting the **negative gradient of whatever loss you chose**. The distinction is not pedantry—it is the entire reason GBDTs can optimize log loss, quantile loss, Poisson deviance, or a ranking objective like LambdaRank [SR 5] with the same tree machinery.

Table 2.1 collects the gradient interface for the losses that actually come up—this is the table to have memorized, because "what are the pseudo-residuals for loss X" is the standard follow-up chain, and the zero-hessian rows explain real library behavior (second-order methods must fall back to unit hessians or per-leaf line search for MAE/quantile objectives).

Table 2.1: The gradient interface: $g_i = \partial L/\partial F(x_i)$, $h_i = \partial^2 L/\partial F(x_i)^2$

Task	**Loss**	$-g_i$ **(pseudo-residual)**	h_i	**Note**
Regression	Squared $\frac{1}{2}(y - F)^2$	$y_i - F(x_i)$	1	The "fit the residuals" special case
Regression	Absolute $\lvert y - F\rvert$	$\text{sign}(y_i - F(x_i))$	0	Fits signs; robust; needs line search
Regression	Huber	clipped residual	1 or 0	Quadratic near, linear far; robust compromise
Classification	Log loss (F = logit)	$y_i - p_i$	$p_i(1 - p_i)$	$p_i = \sigma(F(x_i))$; hessian $\leq 1/4$
Counts	Poisson ($F = \log \mu$)	$y_i - \mu_i$	μ_i	$\mu_i = e^{F(x_i)}$; insurance/demand workhorse
Quantiles	Pinball at τ	$\tau - \mathbb{1}[y_i < F(x_i)]$	0	Prediction intervals from GBDTs (Chapter 4)
Ranking	LambdaRank	λ_i (NDCG-weighted pairs)	pair-based	LambdaMART [SR 5]

Why the weak learners are shallow. A depth-D tree can represent interactions among at most D features along any root-to-leaf path, so tree depth is an *interaction-order* dial: stumps ($D = 1$) yield a purely additive model (a GAM fit by boosting), depth 2 admits pairwise interactions, and so on. Most tabular signal lives at low interaction order, which is why depth 3–8 covers almost every problem and why cranking depth mostly buys variance. This "interaction depth" reading is the substantive answer to "why does boosting use shallow trees while RF uses deep ones"—it is not a compute shortcut, it is a bias budget allocated one

interaction order at a time.

AdaBoost, in one paragraph. AdaBoost (Freund & Schapire, 1997) predates this view and is recovered as forward stagewise minimization of the *exponential loss* $L = e^{-yF}$, $y \in \{-1, +1\}$: the stagewise algebra turns into the familiar recipe of reweighting misclassified samples ($w_i \propto e^{-y_i F(x_i)}$) and combining stumps with weights $\alpha_m = \frac{1}{2}\log\frac{1-\mathrm{err}_m}{\mathrm{err}_m}$. The correct modern framing for the inevitable follow-up: AdaBoost is gradient boosting with a specific (exponential) loss, discovered before the general theory (Friedman, Hastie & Tibshirani, 2000); the exponential loss's aggressive weighting of hard examples also explains AdaBoost's outlier sensitivity, and log loss is preferred today.

2.3.3 Shrinkage and Stochastic Subsampling

Shrinkage. The learning rate $\eta \in (0, 1]$ scales every tree's contribution. Small η with proportionally more rounds almost always generalizes better than large η with few rounds: each step corrects less, leaving more of the signal to be re-examined by later trees under fresh subsamples—a regularization effect empirically robust since Friedman (2001). The operational consequence is that η and M (`n_estimators`) are **one joint knob**: halve η, roughly double M, and always let early stopping on a validation set choose M rather than tuning it directly (Section 2.7).

Stochastic gradient boosting (Friedman, 2002): fit each tree on a random subsample of rows (without replacement, typically 50–80%). This decorrelates consecutive trees, adds variance reduction reminiscent of bagging, and speeds training; column subsampling per tree/level/split does the same on the feature side. Modern libraries expose both (`subsample`, `colsample_*` / `feature_fraction`).

2.3.4 Second-Order (Newton) Boosting

First-order boosting fits the gradient; nothing stops us from using curvature too. Define, per sample,

$$g_i = \left.\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right|_{F_{m-1}}, \qquad h_i = \left.\frac{\partial^2 L(y_i, F(x_i))}{\partial F(x_i)^2}\right|_{F_{m-1}}. \tag{2.12}$$

A second-order Taylor expansion of the stagewise objective turns each round into a **weighted least-squares** problem: minimize $\sum_i \frac{1}{2} h_i \big(f(x_i) - (-g_i/h_i)\big)^2 + \text{const}$, i.e., fit the Newton step $-g_i/h_i$ with sample weights h_i. For log loss, $g_i = p_i - y_i$ and $h_i = p_i(1-p_i)$: confidently predicted samples get tiny hessians and correspondingly little say in where splits go. This Newton view—LogitBoost was the early instance—is the mathematical foundation of XGBoost, where it is combined with explicit regularization and solved in closed form per leaf. That derivation is the next section, and it is the single most-asked whiteboard exercise in the classical-ML canon.

Boosting Attacks Bias; Bagging Attacks Variance

A boosted ensemble is a sum of *shallow, high-bias* trees, each correcting the systematic errors of the sum so far: sequential error-fixing drives bias down, and with enough rounds the ensemble can fit anything—**including noise, so boosting overfits in** M and demands early stopping and shrinkage. A random forest is an average of *deep, low-bias, high-variance* trees: averaging drives variance down and **cannot overfit in** B. This asymmetry—which knob fights which error term, and which ensemble can be run "too long"—is the cleanest bias-variance instantiation in classical ML ([DL 2] for the decomposition itself) and the backbone of the random-forest-vs-GBDT interview answer.

2.4 XGBoost Internals

XGBoost (Chen & Guestrin, 2016) is Newton boosting with regularization built into the objective, plus a set of systems contributions (sparsity-aware split finding, weighted quantile sketch, cache-aware columnar layout) that made it the tool that won half a decade of Kaggle and became the tabular workhorse. Interviews at senior levels reliably probe the objective derivation—it is short, closed-form, and cleanly separates "used XGBoost" from "understands XGBoost."

2.4.1 The Regularized Objective

At round m, with predictions $\hat{y}_i^{(m-1)} = F_{m-1}(x_i)$ frozen, XGBoost seeks the tree f minimizing

$$\mathcal{L}^{(m)} = \sum_{i=1}^{n} L\big(y_i,\, \hat{y}_i^{(m-1)} + f(x_i)\big) + \Omega(f), \qquad \Omega(f) = \gamma T + \frac{\lambda}{2}\sum_{j=1}^{T} w_j^2 \;\big(+\; \alpha\|w\|_1\big), \tag{2.13}$$

where the tree is parameterized by its structure $q : \mathbb{R}^d \to \{1, \dots, T\}$ (which leaf each point falls in) and its leaf weights $w \in \mathbb{R}^T$, so $f(x) = w_{q(x)}$. The complexity penalty is *part of the training objective*, not a bolt-on: γ charges for each leaf, λ (L2) shrinks leaf values, α (L1) can zero them. This is the first structural difference from classic gradient boosting, which regularizes only implicitly (shrinkage, depth limits).

The Classic Derivation: Optimal Leaf Weights and Split Gain

Step 1 — Taylor-expand the loss to second order around the current predictions:

$$L\big(y_i, \hat{y}_i^{(m-1)} + f(x_i)\big) \approx L\big(y_i, \hat{y}_i^{(m-1)}\big) + g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2, \tag{2.14}$$

with g_i, h_i the first and second derivatives of the loss w.r.t. the prediction. Dropping the constant first term:

$$\tilde{\mathcal{L}}^{(m)} = \sum_{i=1}^{n} \Big[g_i f(x_i) + \frac{1}{2} h_i f(x_i)^2 \Big] + \gamma T + \frac{\lambda}{2}\sum_{j=1}^{T} w_j^2. \tag{2.15}$$

Step 2 — Group by leaf. Let $I_j = \{i : q(x_i) = j\}$, $G_j = \sum_{i \in I_j} g_i$, $H_j = \sum_{i \in I_j} h_i$. Since $f(x_i) = w_j$ for all $i \in I_j$:

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^{T} \Big[G_j w_j + \frac{1}{2}\big(H_j + \lambda\big) w_j^2 \Big] + \gamma T. \tag{2.16}$$

The objective has **decoupled into T independent one-dimensional quadratics** in the w_j.

Step 3 — Minimize each quadratic. Setting $\partial / \partial w_j = G_j + (H_j + \lambda) w_j = 0$:

$$\boxed{w_j^* = -\frac{G_j}{H_j + \lambda}} \qquad \text{giving} \qquad \tilde{\mathcal{L}}^{(m)}(q) = -\frac{1}{2}\sum_{j=1}^{T} \frac{G_j^2}{H_j + \lambda} + \gamma T. \tag{2.17}$$

The quantity $\frac{G_j^2}{H_j + \lambda}$ is the **structure score** of a leaf: how much loss reduction the leaf buys. It depends only on sums of gradients and hessians—one reason the whole algorithm vectorizes and distributes so well.

Step 4 — Score a split. Splitting a node with statistics (G, H) into children (G_L, H_L) and (G_R, H_R) ($G = G_L + G_R$, $H = H_L + H_R$) changes the objective by

$$\boxed{\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right] - \gamma} \tag{2.18}$$

Grow the tree by taking the split with the largest gain at each node; a split with Gain < 0 is not worth its γ price—cost-complexity pruning re-derived inside the boosting objective. **Sanity checks worth saying aloud.** For squared loss ($g_i = \hat{y}_i - y_i$, $h_i = 1$): $w_j^* = \frac{\sum_{i \in I_j}(y_i - \hat{y}_i)}{|I_j|+\lambda}$—the mean residual, ridge-shrunk toward zero; with $\lambda = 0$ classic residual-fitting TreeBoost is recovered exactly. For log loss: $g_i = p_i - y_i$, $h_i = p_i(1-p_i)$, and w_j^* is a regularized Newton step in logit space.

2.4.2 Reading the Formulas Like an Interviewer

- **λ acts through every leaf value and every gain.** Larger λ shrinks w^* and deflates structure scores of small-H leaves the most—leaves supported by little (hessian-weighted) evidence.
- **γ is a split admission fee**, directly comparable to gain units; it is the modern α of cost-complexity pruning.
- **`min_child_weight` is a hessian floor, not a sample count.** It requires $H_{\text{child}} \geq$ threshold. For squared loss $h_i = 1$, so it literally is a minimum child size; for log loss $h_i = p_i(1-p_i) \leq 1/4$, so regions where the model is already confident have small hessian mass and get split-blocked early. That is the intended behavior—confident regions provide almost no curvature information, and splits there fit noise—and it is why the same `min_child_weight` value means very different things across objectives (a favorite follow-up).

2.4.3 Split Finding: Exact Greedy vs. Approximate

Exact greedy: for each node and feature, scan all sorted feature values accumulating (G_L, H_L) and evaluate the gain at each boundary—the CART sweep of Section 2.1 with (g, h) sums instead of class counts. Exact but expensive at scale and awkward to distribute.

Approximate split finding: propose a small candidate-threshold set ("global" per tree, or "local" per node) and bucket samples into it. The subtlety is how to choose candidates: plain quantiles of the feature are wrong because samples are not equally important to the second-order objective. Since round-m training is weighted least squares with weights h_i (Section 2.3), XGBoost uses a **weighted quantile sketch**: candidates are equi-mass quantiles under the *hessian* distribution, computed by a mergeable, prunable sketch structure with provable approximation bounds so it works on distributed shards. One sentence of intuition suffices in interviews: "split candidates should be evenly spaced in evidence, not in feature value, and h_i is the evidence weight the Taylor expansion hands you."

Sparsity-aware default directions. Real tabular matrices are full of missing values, zeros from one-hot blocks, and sparse counts. XGBoost scans only the *non-missing* values of a feature—twice, once assigning all missing-value samples to the left child and once to the right—and keeps the better **default direction** per split. Cost becomes linear in non-missing entries

(a huge win on sparse data), and missingness itself becomes learnable signal. At prediction time, a missing value simply follows the stored default arrow.

Systems, in one paragraph. XGBoost stores data in compressed column (CSC) blocks pre-sorted by feature value once before training, so split scans are sequential reads rather than re-sorts; gradient lookups from sorted feature order are cache-hostile, so scans prefetch gradient statistics into per-thread buffers; blocks shard across disk/machines for out-of-core and distributed training. None of this changes the math—it is why the math runs fast—and modern XGBoost defaults to the histogram method (`tree_method=hist`), adopting the binning strategy that LightGBM proved out, which is the next section.

2.4.4 One Boosting Round, End to End

Worth being able to narrate as a closed loop—it is also the skeleton of the coding-round exercise (Chapter 8):

1. From current predictions $\hat{y}^{(m-1)}$, compute g_i, h_i for every sample (one vectorized pass; for log loss, $p_i = \sigma(\hat{y}_i)$, $g_i = p_i - y_i$, $h_i = p_i(1 - p_i)$).
2. Optionally subsample rows (`subsample`/GOSS) and columns for this round.
3. Grow the tree greedily: at each node, for each (sampled) feature, scan candidate thresholds—exact sorted sweep or histogram bins—accumulating (G_L, H_L) and scoring the gain formula; respect `min_child_weight` (H floors) and learn the missing-value default direction; recurse until depth/leaf limits.
4. Prune: drop splits whose gain does not cover γ (bottom-up for depth-wise growth).
5. Set leaf values $w_j^* = -G_j/(H_j + \lambda)$ and update $\hat{y}^{(m)} = \hat{y}^{(m-1)} + \eta\, w_{q(x)}$.
6. Score the validation set; early stopping ends training when it stops improving.

Two loops, a sort or histogram, and a ratio—the entire mystique of the tool that wins tabular ML reduces to this, which is exactly why interviewers feel entitled to ask for it from memory.

2.5 LightGBM

LightGBM (Ke et al., 2017) keeps XGBoost's second-order objective and changes the engineering around it: histogram split finding, leaf-wise growth, and two sampling tricks (GOSS, EFB) aimed at very large n and very wide, sparse d.

2.5.1 Histogram-Based Split Finding

Discretize each feature once, up front, into at most `max_bin` buckets (default 255, so a bin index fits in one byte). Split finding at a node then has two phases:

- **Build**: one pass over the node's samples accumulating $(\sum g, \sum h)$ per bin — $O(n \cdot d)$ with tiny constants (byte reads, no sort).
- **Scan**: evaluate the gain at each bin boundary — $O(\texttt{bins} \cdot d)$, independent of n.

Two multipliers make this even better. First, the **histogram subtraction trick**: a node's histogram equals its parent's minus its sibling's, so only the *smaller* child ever needs a build pass—the larger child's histograms come from a subtraction at $O(\texttt{bins} \cdot d)$. Second, binning shrinks memory (uint8 bins vs. sorted 32-bit indices), which is often the difference between in-memory and out-of-core training. The accuracy cost of coarse thresholds is negligible in

practice—boosting's later trees mop up quantization error, and 255 thresholds per feature is plenty.

2.5.2 Leaf-Wise Growth with a Depth Guard

Classic XGBoost grows **level-wise** (depth-wise): expand every node at the current depth, yielding balanced trees. LightGBM grows **leaf-wise** (best-first): repeatedly split the single leaf with the largest gain anywhere in the tree, until `num_leaves` is reached. For a fixed leaf budget, leaf-wise growth achieves lower training loss—it spends its budget where the gain is, producing deep, lopsided trees that chase structure (and, on small or noisy data, chase noise). Hence the standard discipline: **`num_leaves` is the capacity knob** (default 31), `max_depth` is a guard against pathological depth, and `min_data_in_leaf` (default 20) keeps lopsided branches honest. The classic failure mode of an untuned LightGBM on 10K rows—train metric superb, validation mediocre—is usually `num_leaves` left large with no leaf-size floor.

2.5.3 GOSS: Gradient-Based One-Side Sampling

Samples with small gradients are already well fit; they contribute little to gain estimates. GOSS keeps the information-dense samples and subsamples the rest:

- Keep the top $a \times 100\%$ of samples by $|g_i|$.
- Randomly sample $b \times 100\%$ of the remaining small-gradient samples.
- Multiply the sampled small-gradient samples' statistics by $\frac{1-a}{b}$ in the gain computation, compensating for the ones dropped so the data distribution (and the split-gain estimates) stay approximately unbiased.

The result is bagging-like speedup concentrated where it is safe. GOSS is off by default (`boosting=goss` enables it); it typically buys 2–3× per-iteration speedups at little to no accuracy cost on large data, and matters less on small data where you would not subsample anyway.

2.5.4 EFB: Exclusive Feature Bundling

Wide sparse data (one-hot blocks, count features) contains many features that are almost never nonzero *simultaneously*—mutually exclusive in the graph-coloring sense. EFB greedily bundles nearly-exclusive features into one composite feature, giving each member its own bin range inside the bundle so the histogram can still tell them apart. Histogram cost drops from $O(n \cdot d)$ to $O(n \cdot \texttt{bundles})$, which on one-hot-heavy data is the difference between "wide" and "narrow." Conceptually: EFB recovers, automatically and lossily-but-boundedly, the compactness that one-hot encoding destroyed.

2.5.5 Native Categorical Splits

LightGBM implements the Fisher sorting trick under the second-order objective: at a node, order a feature's categories by $\sum g / \sum h$ (their gradient-statistics "mean response"), then scan that ordering for the best split into two category sets—an optimal-partition search in $O(k \log k)$ rather than $2^{k-1} - 1$. Guards against high-cardinality overfitting: `max_cat_threshold` caps the partition search, `cat_smooth` shrinks per-category statistics, `min_data_per_group` floors category support. Native splits usually beat one-hot for $k \gtrsim 10$ and are far cheaper; at very

high cardinality (user IDs), target-statistics encoding or embeddings remain the better tool (Section 2.6, Chapter 5).

2.5.6 LightGBM vs. XGBoost in Practice

The 2026-honest summary: **accuracy differences are small and dataset-dependent; the choice is about speed, features, and ecosystem.** LightGBM typically trains fastest at large n/wide d and popularized the histogram + leaf-wise combination; XGBoost has since adopted histograms (`hist`, GPU variants) and closed most of the speed gap, retains more conservative level-wise defaults (harder to overfit out of the box), and has the deepest ecosystem integration (Spark, Ray, SageMaker, every serving stack). Interview-safe guidance: on large tabular data default to LightGBM for iteration speed; on small data prefer conservative settings whichever library you use; treat a claimed accuracy edge of either as noise until proven on *your* data; and consider CatBoost when categoricals dominate—next section.

2.6 CatBoost

CatBoost (Prokhorenkova et al., 2018) is the third member of the trinity, built around one statistical idea—**both categorical encoding and boosting itself leak the target if a row's own label influences the statistics used to fit that row**—and one architectural one (oblivious trees).

2.6.1 Target Statistics and the Leakage Problem

Target encoding replaces category c with a statistic of the target over rows in that category, typically smoothed toward the global prior P:

$$\hat{x}_i = \frac{\sum_{j \neq i} \mathbb{1}[c_j = c_i]\, y_j + aP}{\sum_{j \neq i} \mathbb{1}[c_j = c_i] + a}. \tag{2.19}$$

It is the only encoding that scales to very high cardinality with no dimensionality cost—and computed naively (over the *full* training set, including row i itself), it is a target-leakage machine.

Warning

Worked mini-example of the leak. Suppose `merchant_id = M1729` appears exactly once in training, on a fraudulent transaction ($y = 1$). Naive unsmoothed target encoding assigns that row the feature value 1.0—*the row's own label, copied into a feature.* Even with smoothing a, the row's encoding is pulled toward its own label; across thousands of rare categories the encoded feature becomes a noisy copy of y, trees split on it immediately, training metrics soar, and validation collapses. The signature is unmistakable: a target-encoded high-cardinality feature at the top of the importance list with a too-good-to-be-true training score. The general taxonomy of such bugs is Chapter 5; the fix used outside CatBoost is out-of-fold encoding (compute row i's statistic from folds not containing i).

2.6.2 Ordered Target Statistics

CatBoost's fix is a streaming version of "only use the past": draw a random permutation σ of the training rows and encode row i using **only the rows that precede it in σ**:

$$\hat{x}_i = \frac{\sum_{\sigma(j)<\sigma(i)} \mathbb{1}[c_j = c_i]\, y_j + aP}{\sum_{\sigma(j)<\sigma(i)} \mathbb{1}[c_j = c_i] + a}. \tag{2.20}$$

Row i's own label can never touch row i's feature—the estimate behaves like an online mean over an artificial "history." Early rows get noisy, prior-dominated encodings, so CatBoost uses several permutations across iterations to average that noise out. It also generates *combinations* of categorical features (greedy, per split) with the same ordered statistics, capturing interactions like region $\times$ device that a single encoding misses.

2.6.3 Ordered Boosting and Prediction Shift

The subtler observation of the CatBoost paper: **plain gradient boosting has the same disease**. The residual/gradient g_i for row i is computed from F_{m-1}—a model that was itself *fit on row i*. Predictions on training rows are systematically closer to their labels than predictions on fresh data would be, so the gradients are biased ("prediction shift"), and each round compounds the bias: the ensemble is slightly overfit by construction, most visibly on small datasets. **Ordered boosting** applies the permutation idea to the boosting loop itself: conceptually, maintain models such that the residual for row i comes from a model trained only on rows preceding i in σ—never on i itself. Maintaining n models is infeasible, so CatBoost approximates with a logarithmic number of support models over a few permutations; the cost is extra training time, which is why ordered boosting is the default only up to moderate dataset sizes (its `Plain` mode is standard boosting). Intuition-level fluency here is enough for interviews: *ordered statistics fix target leakage through features; ordered boosting fixes target leakage through the residuals.*

2.6.4 Oblivious (Symmetric) Trees

CatBoost's base learner uses the **same split (feature, threshold) across an entire level**: a depth-D oblivious tree is a sequence of D binary tests, and every sample's leaf is just the D-bit vector of test outcomes indexing a 2^D-entry table. Consequences: each tree is a weaker, heavily regularized learner (good for generalization, more rounds needed); and inference is spectacular—no per-node branching, just D comparisons and a table lookup, vectorizing across trees and samples—which is why CatBoost routinely wins pure model-inference latency benchmarks among the three libraries.

2.6.5 When CatBoost Wins

Choose CatBoost when categorical features carry the signal—especially many mid-to-high-cardinality ones—on small-to-mid data where leakage and prediction shift bite hardest, or when you want strong accuracy with minimal tuning (its defaults, including an auto-set learning rate, are the most robust of the three). Costs: typically the slowest training of the trio at large scale, and a smaller ecosystem. Table 2.2 summarizes the trinity.

Table 2.2: The GBDT trinity at a glance

Dimension	**XGBoost**	**LightGBM**	**CatBoost**
Tree growth	Level-wise default (leaf-wise available)	Leaf-wise, `num_leaves` capacity	Oblivious (symmetric) trees
Split finding	Exact greedy or histogram; weighted quantile sketch	Histogram (binned, subtraction trick)	Histogram on top of ordered statistics
Categoricals	Native support recent; commonly encoded upstream	Native $\sum g / \sum h$-sorted optimal partitions	Ordered target statistics + feature combinations (the headline feature)
Overfitting posture	Conservative defaults; explicit γ, λ, α	Fastest, most eager; needs care with `num_leaves` and `min_data_in_leaf`	Most regularized by construction (ordered boosting, symmetric trees)
Speed profile	Fast; deepest distributed/ecosystem support	Usually fastest training at scale (GOSS, EFB)	Slowest training; fastest model inference
Pick when	Default ecosystem choice; ranking objectives mature [SR 5]	Large n or wide sparse d; iteration speed matters	Heavy categoricals; small-mid data; minimal tuning budget

2.7 Hyperparameter Craft

GBDTs expose dozens of parameters; roughly eight matter, and interviewers care less about the list than about whether you know *which direction each one moves the bias-variance needle* and *in what order to touch them.*

Table 2.3: The eight knobs that matter (XGBoost / LightGBM names)

Knob	**What it controls**	**Overfitting? Move it...**
`n_estimators` + early stopping	Number of boosting rounds M	Set M high, let early stopping on a validation set choose; never hand-tune
`learning_rate` (η)	Step size per round; joint knob with M	Down (with more rounds); 0.02–0.1 typical for final models
`max_depth` / `num_leaves`	Per-tree capacity (interaction order)	Down; depth 3–8 / leaves 15–127 cover most problems
`min_child_weight` / `min_data_in_leaf`	Evidence floor per leaf (hessian mass / row count)	Up; blocks noise-chasing splits
`subsample` / `bagging_fraction`	Row sampling per round (stochastic GB)	Down toward 0.6–0.8; adds decorrelation
`colsample_bytree` / `feature_fraction`	Column sampling	Down toward 0.6–0.8; the RF trick inside boosting
λ, α (`reg_lambda`, `reg_alpha`), γ	Leaf-value shrinkage (L2/L1), split admission fee	Up; acts inside every gain via Section 2.4
Categorical handling	Native vs. one-hot vs. target/ordered statistics	Not a dial but a decision; wrong choice dominates all other tuning on categorical-heavy data

A sane tuning order. (1) Fix $\eta \approx 0.05$–0.1, `n_estimators` large, early stopping on (say 50–200 rounds patience). (2) Tune capacity: `max_depth`/`num_leaves` together with the leaf floor `min_child_weight`/`min_data_in_leaf`—this pair moves validation metrics most. (3) Tune `subsample` and `colsample` (coarse grid, 0.6–1.0). (4) Nudge λ/γ if the train-validation gap is still wide. (5) For the final model, halve η, refit with early stopping, and stop—returns beyond this are usually noise. Random or Bayesian search over this space beats grid search (Chapter 5); and any tuning claim needs an honest validation protocol, or the winner's curse eats the reported gain.

The overfitting-diagnosis loop. This is bias-variance triage ([DL 2] owns the decomposition; here it is instantiated):

- **Train metric excellent, validation flat or degrading (variance problem—the common case):** early stopping first (it is the fix for "got worse after round k" by construction); then capacity down, leaf floors up, subsampling on, regularization up, η down with more rounds.
- **Train and validation both poor (bias problem):** capacity up, leaf floors down, more rounds at lower η; if the ceiling persists, the model is not the bottleneck—engineer features (interactions trees cannot see, e.g., ratios; better encodings) or question the labels.
- **Validation great, production poor:** not a hyperparameter problem—suspect leakage or drift before touching a knob (Chapter 5, [DL 22]).

2.8 GBDT vs. Neural Networks on Tabular Data

The full decision-framework interview question lives in the deep-learning volume [DL 24]; this section supplies the classical-side mechanism—*why* the inductive bias of trees fits tabular data—plus the 2024–26 status update.

2.8.1 The Inductive-Bias Story

Grinsztajn et al. (2022, NeurIPS Datasets & Benchmarks) ran the cleanest controlled comparison to date (45 mid-size tabular datasets, tuned baselines on both sides: XGBoost/RF vs. MLP/ResNet/FT-Transformer) and, more usefully, isolated **three mechanisms** for why tuned GBDTs still won on medium data:

1. **Tabular target functions are irregular.** NN training is biased toward smooth functions (a feature, not a bug, for images and audio); tabular targets are full of sharp thresholds and plateaus. Smoothing the targets hurt trees and barely affected NNs—evidence the un-smooth component is real signal that piecewise-constant splits capture natively.
2. **Uninformative features hurt NNs more.** Tabular datasets carry many weakly-or-uninformative columns. Removing them narrowed the tree-NN gap; adding noise features widened it. Trees simply stop splitting on useless features; MLP-class models must learn to ignore them in every direction of a rotating representation.
3. **Rotation invariance is the wrong symmetry.** An MLP's first layer sees the data only up to rotation of feature space; randomly rotating tabular features leaves MLP performance roughly unchanged but degrades trees badly—and after rotation, NNs come out *ahead*. Read correctly, this says the original feature axes carry meaning (`age`, `income`, `days_since_last_login` are individually semantic), and the tree's axis-aligned "bias" is precisely the alignment with how tabular data is generated. Trees are **not** rotation-invariant, and on tables that is a feature.

Inductive-Bias Match Beats Capacity

Nothing about tables makes them "too hard" for neural networks—the point is the opposite. NNs bring smoothness and rotation invariance as priors; tabular data offers irregular, axis-aligned structure with junk columns, heterogeneous scales, and missing values. GBDTs bring exactly matching priors (piecewise-constant, axis-aligned, monotone-transform-invariant, split-level feature selection, native missingness) plus a training procedure that is minutes-fast and hard to destabilize. When the modality changes—raw text, images, sequences, or IDs needing learned embeddings—the matching prior flips sides, and so does the winner. Framing the question as "which model is stronger" rather than "whose prior matches the data" is itself the junior tell.

2.8.2 Pragmatics the Benchmarks Undersell

Real tabular pipelines tilt further toward GBDTs than accuracy deltas suggest: heterogeneous types and NaNs work with zero preprocessing (no imputation, no scaling, no embedding plumbing); training is minutes on CPUs (no GPU queue, cheap retrains under drift); hyperparameters fail gracefully (a bad GBDT config underperforms; a bad NN config diverges); and small-team maintainability favors the two-hundred-line training script. These operational facts, not bench-

mark wins, are why fraud, credit, pricing, and churn systems at platform companies still default to GBDTs in 2026.

2.8.3 The 2024–26 Status of Tabular Deep Learning

An honest update, because "GBDTs always win on tables" is now stale:

- **Small data has genuinely shifted.** TabPFN v2 (Hollmann et al., *Nature* 2025) is a transformer pre-trained on synthetic tabular tasks that performs *in-context* prediction—no gradient training on your dataset at all—and outperforms tuned GBDTs on many small datasets (up to roughly 10K rows / 500 features), in seconds. In-context tabular learners are real competition in the small-data regime and a strong name-drop with follow-up risk: know that it is a prior-fitted network, that inference cost grows with the training set carried in context, and that the regime cap is the point.
- **Mid-scale tables: GBDT remains the default.** Post-2022 re-evaluations (e.g., McElfresh et al., 2023) soften the headline to "dataset-dependent, GBDTs win more often and are vastly cheaper to get right"; improved NN recipes (FT-Transformer descendants, ensemble-style MLPs such as TabM) narrow but do not close the mid-scale gap, and cost 10–100× more tuning compute to reach parity.
- **NNs win with cause**: very large data with dense feature interactions; high-cardinality IDs where learned embeddings are the representation (the recommender regime—[SR 5] and the wide ranking literature); multimodal inputs (table + text + image); online/continual learning where incremental gradient updates beat periodic retrains; or when the tabular signal feeds a larger neural system anyway.
- **The practical ensemble-of-both pattern**: blending a GBDT and a tabular NN reliably adds a small gain (their errors decorrelate—exactly the ρ logic of Section 2.2); AutoML stacks (AutoGluon-style) institutionalize this.

2.9 Interpretation: SHAP, TreeSHAP, Partial Dependence

2.9.1 Shapley Values and TreeSHAP

SHAP (Lundberg & Lee, 2017) attributes a single prediction to features using the Shapley value from cooperative game theory: feature j's attribution ϕ_j is its contribution to the prediction averaged over all orders in which features could be "revealed," with the model's expectation over hidden features filling in for the unrevealed ones. Two properties make it the standard:

- **Efficiency/additivity**: $\sum_j \phi_j = f(x) - \mathbb{E}[f(X)]$—attributions exactly decompose the prediction's deviation from the baseline, so they can be summed, compared, and audited.
- **Consistency**: if a model changes so a feature matters more everywhere, its attribution cannot decrease (the axiom MDI violates).

Exact Shapley values cost $O(2^d)$ in general—but trees make the coalition expectations computable by dynamic programming over paths. **TreeSHAP** (Lundberg et al., 2020) computes exact SHAP values for tree ensembles in polynomial time—$O(TLD^2)$ per prediction for T trees, L leaves, depth D—which is *the* reason SHAP-based explanation is ubiquitous for GBDTs and comparatively rare (approximate, sampled) for NNs. Global importance is the mean $|\phi_j|$ over a dataset; summary (beeswarm) plots add direction and distribution.

2.9.2 Caveats That Interviewers Probe

The additivity guarantee is about the *model*, not the world. The three standard traps: **(1) bar plots are interaction-blind**—mean-$|\phi|$ rankings collapse a feature whose effect flips sign across segments into a mid-table entry (dependence/interaction plots exist for this); **(2) correlated features dilute and redistribute credit**, and the choice between interventional and observational (conditional) expectations changes answers precisely when correlation is strong—off-manifold evaluation again; **(3) SHAP is not causal**: a high ϕ for `discount_rate` on churn says the model uses it, not that changing it changes outcomes. The full trap catalog, including background-set sensitivity and retrain instability, is centralized in Chapter 5.

Partial dependence, one paragraph. A PDP plots the prediction averaged over the data as one feature is swept across its range—the model-level analogue of a dose-response curve. It assumes the swept feature is independent of the rest (the sweep creates impossible combinations otherwise), and averaging hides heterogeneous effects; ICE curves (one line per sample) reveal what the average conceals. PDPs remain the quickest sanity check that a fitted GBDT's shape matches domain knowledge—and when the shape *must* match, enforce it instead: monotonic constraints, next section.

2.10 Production Notes

Size and latency: trees vs. depth. Model size is dominated by leaf count: T trees $\times$ L leaves, each internal node storing (feature id, threshold, child pointers), each leaf a value—a 1,000-tree, 64-leaf model is a few MB, trivially cacheable. Inference cost is $T\times$ (average depth) branchy comparisons per row; the trade that matters is that **many shallow trees beat few deep trees at equal accuracy for latency**, because shallow trees vectorize and mispredict branches less—and lower η/more trees pushes the other way, so serving-constrained teams tune (T, depth, η) jointly against a latency budget. CatBoost's oblivious trees are the extreme point: pure table lookups, typically the fastest inference of the trio.

Compilation, one-liner. Do not serve GBDTs through a Python training API: compile the model—Treelite/tl2cgen or lleaves (LLVM) for optimized native code, or export to ONNX for runtime-agnostic serving—for typically 2–10$\times$ single-row latency improvements; and in real systems, profile end-to-end first, since feature fetching usually dwarfs model compute [DL 22].

Monotonic constraints. XGBoost/LightGBM/CatBoost can constrain the prediction to be monotone in chosen features by refusing violating splits (enforced at split time: a constrained feature's split is admitted only if child values respect the direction). Use them when policy or physics demands it—credit score vs. probability of approval, price vs. demand—accepting a small accuracy cost for guaranteed shape, audit-friendliness, and robustness to sparse regions. This is "interpretability by construction" and a strong senior signal to volunteer.

Calibration pointer. Boosted-tree scores optimized with early stopping on log loss are reasonably calibrated but drift toward extremes with heavy rounds, and class weights or resampling distort them badly; random-forest vote fractions are compressed away from 0 and 1. If downstream consumers threshold on probabilities, calibrate and monitor—mechanics in Chapter 4 and [DL 23].

Ranking pointer. The dominant industrial learning-to-rank method, LambdaMART, is exactly this chapter's machinery with (g_i, h_i) supplied by LambdaRank's NDCG-weighted pairwise gradients—the GBDT is the optimizer, the ranking loss is plugged in through the gradient interface of Section 2.3. Full treatment in [SR 5].

2.11 Interview Questions

Interview Question

Q: Derive the optimal leaf weight and the split-gain formula in XGBoost.

L5 Mathematical ★★★

Strong answer:

Set up the round-m objective: loss over frozen predictions plus a new tree f, with regularization $\gamma T + \frac{\lambda}{2}\sum_j w_j^2$. Taylor-expand the loss to second order in $f(x_i)$, defining g_i and h_i as the first and second derivatives at the current prediction, and drop the constant term. Then group the sum by leaf: with $G_j = \sum_{i \in I_j} g_i$ and $H_j = \sum_{i \in I_j} h_i$, the objective becomes $\sum_j [G_j w_j + \frac{1}{2}(H_j + \lambda) w_j^2] + \gamma T$—$T$ independent quadratics. Each is minimized at

$$w_j^* = -\frac{G_j}{H_j + \lambda}, \qquad \text{leaving objective } -\frac{1}{2}\sum_j \frac{G_j^2}{H_j + \lambda} + \gamma T.$$

A split's value is the change in this objective:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma.$$

Close with the sanity check: for squared loss ($h_i = 1$), w^* is the ridge-shrunk mean residual; for log loss, $g = p - y$, $h = p(1-p)$, and w^* is a regularized Newton step. Note what the derivation buys: regularization inside the objective, γ as principled pruning, and split scoring from (G, H) sums alone—which is why it distributes.

Red flags (what NOT to say):

- Writing the first-order expansion only—the closed form does not exist without h_i, and "XGBoost is just gradient boosting but faster" misses the Newton objective entirely.
- Forgetting λ in the denominator or γ in the gain, i.e., reciting an unregularized formula for the library whose selling point is regularization.
- Unable to say what g_i and h_i are for log loss.

What the interviewer is testing: Whether you can execute a short constrained-optimization derivation about a tool you claim daily fluency with. The grouping-by-leaf step (sum over samples → sum over leaves) is where shaky candidates stall.

What separates good from great:

- L5 Clean derivation to both boxed formulas with correct g, h for log loss.
- L6 Adds the sanity checks, explains γ as in-objective cost-complexity pruning and `min_child_weight` as a hessian floor, and notes the weighted-least-squares reading (weights h_i, targets $-g_i/h_i$) that justifies the weighted quantile sketch.
- L7 Connects the derivation to systems consequences (gain from (G, H) sums ⇒ histograms, distribution, GPU) and to LambdaMART, where custom (g, h) plug ranking losses into the same solver.

Common follow-ups: "What changes with L1 (α) on leaf weights?" "Why is `min_child_weight` different for regression vs. classification?" "Where does the histogram method approximate this derivation, if anywhere?"

Interview Question

Q: Random forest vs. gradient boosting—how do they differ, and when do you reach for each?

L5 Conceptual ★★★

Strong answer:

They are opposite answers to the single-tree variance problem. **RF**: average B deep, independently perturbed (bootstrap + feature-subsampled) low-bias trees; variance falls as $\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, bias stays put—so RF *reduces variance*, trains in parallel, cannot overfit in B, and gives OOB estimates for free. **GBDT**: sequentially add shallow high-bias trees, each fit to the negative gradient of the loss at current predictions—so boosting *reduces bias*, must train sequentially, and overfits in rounds, demanding shrinkage and early stopping. When each: RF when you want a strong, nearly tuning-free baseline, robustness on small/noisy data, free validation via OOB, and trivially parallel training; GBDT when you want peak tabular accuracy, custom losses (quantile, Poisson, ranking), monotonic constraints, and can pay the tuning/early-stopping discipline. In 2026 practice: RF is the first-hour baseline; a tuned LightGBM/XGBoost/CatBoost is the production model; the gap is usually a few points in the GBDT's favor.

Red flags (what NOT to say):

- Inverting the mechanism: "RF reduces bias" / "boosting reduces variance."
- "More trees in a random forest overfits"—confusing B with boosting rounds M.
- Unable to state why RF trees are deep while GBDT trees are shallow (each ensemble needs its base learner's error profile: low-bias for averaging, high-bias for sequential correction).

What the interviewer is testing: Whether ensembles are organized in your head by *which error term they attack*, not as two library names. The deep-vs-shallow base learner question is the standard depth probe.

What separates good from great:

- L5 Correct variance-vs-bias framing, parallel-vs-sequential, overfitting-in-B vs. -in-M.
- L6 Writes the correlated-variance decomposition and uses it to explain `max_features`; mentions OOB mechanics and RF's low tunability vs. GBDT's high tunability.
- L7 Notes stochastic GBDT imports RF's tricks (row/column subsampling) so modern GBDTs attack both error terms, and discusses when RF still wins (tiny noisy data, no validation set to stop on, massively parallel training budget).

Common follow-ups: "Derive the variance of an average of correlated estimators." "Why does feature subsampling help RF but column subsampling is optional for GBDT?" "Which is more robust to label noise, and why?"

Interview Question

Q: Explain gradient boosting to someone who already understands gradient descent.

L5 First Principles ★★★

Strong answer:

Gradient descent updates a parameter vector by stepping against the gradient of the loss. Gradient boosting does the same thing where the "parameter vector" is *the model's predictions at the training points*—gradient descent in function space. Each round: (1) compute the negative gradient of the loss with respect to each prediction (the pseudo-residuals $r_i = -\partial L/\partial F(x_i)$); (2) you cannot apply that step directly—it is only defined at n points and would not generalize—so *project it onto a function class* by fitting a small tree to the r_i by least squares; (3) update $F \leftarrow F + \eta f$, with the learning rate η as the step size. "Fit the residuals" is what this reduces to under squared loss, where $r_i = y_i - F(x_i)$; under log loss the tree fits $y_i - p_i$; under a ranking loss it fits Lambda gradients [SR 5]. The tree is the descent direction; boosting rounds are iterations; early stopping is the convergence check against a validation set.

Red flags (what NOT to say):

- Presenting residual-fitting as the definition rather than the squared-loss special case.
- No answer for why the raw gradient step must be replaced by a fitted tree (the generalization/projection point).
- Confusing the learning rate on leaf values with the gradient-descent learning rate on parameters inside some tree (there is no such thing—trees are fit greedily, not by SGD).

What the interviewer is testing: Whether you hold the functional-gradient abstraction or a memorized recipe. The projection step—why a tree stands in for the gradient—is the discriminator.

What separates good from great:

- L5 Clean function-space analogy with the squared-loss and log-loss special cases.
- L6 Adds the second-order upgrade (Newton step $-g/h$ with hessian weights; XGBoost's closed-form leaves) and shrinkage-vs-rounds as step-size-vs-iterations.
- L7 Places AdaBoost as the exponential-loss special case, and explains what the gradient interface buys in production: any differentiable loss—quantile, Poisson, LambdaRank—rides the same tree engine.

Common follow-ups: "What are the pseudo-residuals for absolute loss, and what does that imply?" "Why does a lower learning rate generalize better?" "How does AdaBoost fit into this picture?"

Interview Question

Q: XGBoost vs. LightGBM vs. CatBoost: 600K rows, 45 features of which 14 are categorical including a 40K-cardinality `merchant_id`. Choose and defend.

L6 Trade-off ★★★

Strong answer:

First name the decision axes: all three share the second-order objective, so accuracy deltas are small and dataset-dependent—the choice turns on **categorical handling, scale/speed, overfitting posture, and ecosystem**. Here the dominant fact is 14 categoricals with one at 40K cardinality. One-hot is out (40K columns of near-useless indicators); naive target encoding is a leakage trap. That points to **CatBoost**: ordered target statistics handle `merchant_id` natively and leak-free, feature combinations capture categorical interactions, and 600K rows is comfortably within its speed envelope. The defensible alternative is **LightGBM** with its native $\sum g / \sum h$-sorted splits for the mid-cardinality categoricals plus a properly out-of-fold target (or frequency) encoding for `merchant_id`—faster iteration, at the cost of owning the encoding hygiene yourself. XGBoost is the fallback if the surrounding platform (Spark pipelines, existing serving) already speaks it; nothing about the data demands it. State the tie-breakers you would actually run: one afternoon, all three with early stopping on an identical temporal validation split, compare metric, training time, and inference latency; expect differences of tenths of a point and pick on operational grounds.

Red flags (what NOT to say):

- One-hot encoding the 40K-cardinality feature without hesitation—the cardinality-explosion red flag.
- Choosing on "X is more accurate" folklore without mentioning categorical handling at all.
- Proposing naive (in-fold) target encoding, or not knowing why CatBoost's version avoids the leak.

What the interviewer is testing: Judgment under a concrete dataset: whether you map data properties to library mechanisms rather than reciting a favorite. The high-cardinality categorical is planted; the interviewer is watching what you do with it.

What separates good from great:

- L5 Picks a reasonable library with a coherent reason touching categoricals.
- L6 Names the leakage mechanism of naive target encoding, CatBoost's ordered fix, LightGBM's sorted-partition trick and its high-cardinality guards, and proposes the empirical bake-off with a temporal split.
- L7 Extends to system context: embedding-based alternatives if a neural stack already exists for `merchant_id`, inference-latency implications (oblivious trees), retraining cadence under merchant churn, and cold-start merchants (prior-dominated encodings degrade gracefully).

Common follow-ups: "What exactly goes wrong if you target-encode with the full training set?" "New merchants appear daily—what happens to each library's encoding?" "Now the dataset is 500M rows—does your answer change?"

Interview Question

Q: Why does a gradient-boosted tree beat your MLP on this tabular dataset—and when would it not?

L6 Conceptual ★★★

Strong answer:

Lead with the mechanism, not the benchmark: it is an inductive-bias match. Three findings from the controlled comparison literature (Grinsztajn et al., 2022): (1) tabular targets are *irregular*—sharp thresholds and plateaus that piecewise-constant splits fit natively while NN training is biased toward smooth functions; (2) tables carry *uninformative features* that trees ignore by never splitting on them, while MLPs must learn to suppress them; (3) MLPs are *rotation-invariant* and tabular data is not—feature axes are individually meaningful, and the tree's axis-aligned "limitation" is exactly the right prior (randomly rotating features barely affects an MLP but wrecks trees, confirming the axes carry the signal). Add the pragmatics: native NaN and mixed types, monotone-transform invariance, minutes of CPU training, graceful hyperparameter failure. Then give the honest flip side: NNs win with heavy learned embeddings (high-cardinality IDs, the recommender regime), multimodal inputs, online/continual updates, very large data with dense interactions—and on *small* tables, in-context learners like TabPFN v2 (Nature 2025) now beat tuned GBDTs outright. GBDT is the mid-scale tabular default, not a law of nature.

Red flags (what NOT to say):

- "NNs are more powerful, it must be a tuning issue"—capacity is not the axis; prior match is.
- No mechanism at all, just "trees are better on tabular" as folklore.
- Unaware the small-data regime has shifted (TabPFN-class models) or that the recommender world is neural for a reason.

What the interviewer is testing: Whether you can explain a well-known empirical regularity with mechanisms, and whether your knowledge has a 2024–26 update attached. The full decision framework is a deep-learning-volume staple [DL 24]; this version drills the "why."

What separates good from great:

- L5 Two or three correct mechanisms plus the standard when-NNs-win list.
- L6 All three Grinsztajn mechanisms with the rotation experiment stated correctly, plus the TabPFN v2 small-data update and the ensemble-of-both pattern.
- L7 Discusses what would change the answer (embeddings as features feeding the GBDT, distillation across the boundary, the operational-cost asymmetry) and treats the boundary as an engineering decision with measurable criteria.

Common follow-ups: "Design the experiment to test whether the NN's problem is the uninformative features." "What is TabPFN actually doing at inference time?" "Your tabular data has one text column—now what?"

Interview Question

Q: Your LambdaMART ranker's training NDCG keeps improving, but validation NDCG peaked at tree 200 and is now degrading. Walk me through your response.

L6 Debugging ★★○

Strong answer:

Diagnose first: monotone train improvement with validation peak-then-decline is textbook **variance overfitting in boosting rounds**—the ensemble is past its optimal M and

now fitting noise (label noise is endemic in ranking data: clicks are noisy relevance). The response, in order: (1) **early stopping** on validation NDCG with patience—this is the fix for the stated symptom by construction; take the tree-200 model today. (2) Rebalance the η–M trade: lower the learning rate, refit with early stopping; if the peak moves later and higher, capacity per round was too aggressive. (3) Reduce per-tree capacity (`num_leaves`/depth down, `min_data_in_leaf` up) and enable row/column subsampling. (4) **Audit the validation protocol before trusting any of it**: ranking data must be split *by query* (and temporally, if queries repeat)—a query with documents in both train and validation is group leakage that makes validation optimistic and the divergence look milder than it is; also confirm the validation query count is large enough that NDCG differences are not noise. (5) If degradation is severe and early, check the objective's metric weighting (NDCG truncation depth) matches the evaluation metric.

Red flags (what NOT to say):

- Not saying "early stopping" in the first breath, or proposing to tune `n_estimators` by grid search.
- Treating it as a mystery rather than expected boosting behavior—this is what overfitting in M looks like.
- No mention of query-level split integrity—the ranking-specific leakage check is the seniority marker here.

What the interviewer is testing: Debugging discipline on the most common GBDT pathology, plus whether you know the ranking-specific traps (query-grouped splits, noisy click labels, NDCG variance). LambdaMART internals live in [SR 5]; this question tests the boosting-side hygiene.

What separates good from great:

- L5 Early stopping plus the standard variance-reduction knob list.
- L6 Adds the validation-protocol audit (query grouping, temporal split, metric noise) before knob-turning, and the η–M joint reasoning.
- L7 Asks what changed if this is a retrain of a previously stable pipeline (data drift, label pipeline bug, position-bias correction change), and quantifies the decision: ship tree-200 now, investigate in parallel.

Common follow-ups: "Why are click labels particularly conducive to overfitting?" "Your validation NDCG is flat across all settings—what do you suspect?" "How would position bias in the training clicks show up here?"

Interview Question

Q: Why do trees split on Gini or entropy instead of misclassification error, since error is what we ultimately care about?

L5 Conceptual ★★○

Strong answer:

Because splitting needs a criterion that rewards *progress toward* purity, not just purity's final cash value. Misclassification error is piecewise linear in the class proportions: any split that does not change a child's majority class can look worthless. Gini and entropy are strictly concave, so making children purer *always* scores positive gain, even when

the argmax class is unchanged. Give the canonical example: parent $(4+, 4-)$; split A $\to (3+, 1-), (1+, 3-)$ and split B $\to (2+, 4-), (2+, 0-)$ have *identical* weighted misclassification error $(1/4)$, but Gini prefers B (0.333 vs. 0.375), which created a pure node—obviously the better tree to keep growing. Greedy growth needs the strictly concave criterion to see one step ahead; held-out error remains the right quantity for *pruning* and evaluation. Gini vs. entropy itself is a non-event (they disagree on a few percent of splits); Gini skips the log.

Red flags (what NOT to say):

- "Gini is more accurate"—it is about strict concavity and split sensitivity, not accuracy.
- Unable to produce any example where misclassification error is indifferent and Gini is not.
- Confusing the splitting criterion with the evaluation metric, or claiming trees minimize accuracy loss directly.

What the interviewer is testing: A concavity-of-the-criterion argument delivered with a concrete two-line example. This is a warm-up question; fumbling it colors the rest of the tree discussion.

What separates good from great:

- L5 The concavity argument plus the worked example.
- L6 Notes Jensen's inequality guarantees nonnegative gain for concave impurities, and that misclassification error still serves pruning; mentions the regression analogue (variance reduction).

Common follow-ups: "When do Gini and entropy actually disagree?" "What is the regression-tree splitting criterion and why does it admit an $O(1)$ update?"

Interview Question

Q: What fraction of the training data does each tree in a random forest see? Derive it, and explain what the rest is used for.

L5 Mathematical ★★○

Strong answer:

A bootstrap sample draws n times with replacement. The probability a given point is missed by one draw is $1-\frac{1}{n}$; by all n draws, $(1-\frac{1}{n})^n$, which converges to $e^{-1} \approx 36.8\%$ (from $(1+x/n)^n \to e^x$ with $x=-1$). So each tree trains on about 63.2% of the unique points; the missed 36.8% are its **out-of-bag** set. Aggregating, for each point, only the $\approx B/e$ trees that never saw it gives OOB predictions—an approximately unbiased generalization estimate with *no held-out data and no extra training*: free cross-validation. Uses: model selection (pick `max_features` by OOB error), ensemble sizing (grow B until OOB error plateaus), and OOB permutation importance. Caveat worth volunteering: OOB error is slightly pessimistic since each OOB prediction ensembles only $\sim B/e$ trees rather than all B.

Red flags (what NOT to say):

- "Bootstrap samples contain almost all the points"—not knowing the $1/e$ result at all.

- Deriving the limit but having no idea what OOB estimation is for.
- Claiming OOB replaces a test set for the *final* performance claim (it is still training-distribution data used during model development).

What the interviewer is testing: A thirty-second limit derivation plus whether you know the free lunch it powers. This is a fluency check—hesitation on $(1 - 1/n)^n$ suggests the bagging story is memorized, not owned.

What separates good from great:

- L5 Correct derivation and the OOB-as-free-validation story.
- L6 Adds the B/e-trees pessimism caveat and OOB permutation importance; notes GBDTs have no OOB analogue (sequential fitting), which is one reason they need explicit validation sets.

Common follow-ups: "Why doesn't gradient boosting have an OOB equivalent?" "How would you use OOB error to size the forest?"

Interview Question

Q: How does CatBoost avoid target leakage with categorical features, and what is "ordered boosting" fixing?

L6 Conceptual ★★○

Strong answer:

Two leaks, one idea. **Leak 1, through features**: target encoding computed over the full training set includes row i's own label in row i's feature—for a singleton category the feature *is* the label, and rare categories generally get label-contaminated encodings that trees exploit, inflating train metrics and collapsing in validation. CatBoost's **ordered target statistics**: draw a random permutation, encode each row using only rows *before* it (smoothed toward the prior), so no row's label ever touches its own feature; multiple permutations average out the noisy early-row encodings. **Leak 2, through residuals**: in any boosting, row i's gradient is computed from a model that was trained on row i, so training-set gradients are systematically biased ("prediction shift"), compounding across rounds—boosting mildly overfits by construction, worst on small data. **Ordered boosting** applies the same permutation discipline to the loop: each row's residual comes (conceptually) from a model trained only on preceding rows, approximated tractably with a small set of support models. The unifying principle—never let an example's label influence the statistics used to fit that example—is the same one behind out-of-fold target encoding and temporal validation splits.

Red flags (what NOT to say):

- Explaining ordered statistics but drawing a blank on ordered boosting (the question names it).
- "Just use label encoding instead"—misses that the entire point of target statistics is scaling to high cardinality.
- Not connecting to the general out-of-fold principle—this signals the fix is memorized trivia.

What the interviewer is testing: Whether you understand leakage as a mechanism (own-

label contamination) rather than a checklist, and can follow it into the less obvious residual pathway. The general leakage taxonomy is a sibling-chapter staple; this is its sharpest instance.

What separates good from great:

- L5 The singleton-category example and the ordered-statistics fix.
- L6 Both leaks, the prediction-shift argument at intuition level, and the out-of-fold unification.
- L7 Discusses the compute trade (why ordered boosting is default only at moderate scale), permutation-count variance, and when plain mode plus out-of-fold encoding is the pragmatic equivalent.

Common follow-ups: "Why multiple permutations?" "How would you get the same protection in LightGBM?" "Does prediction shift matter at 100M rows?"

Interview Question

Q: Why can't a tree-based model extrapolate, and where does that bite in practice?

L5 Conceptual ★★○

Strong answer:

A tree predicts a constant per leaf, and every leaf value is a statistic (mean, vote, shrunk Newton step) of *training* targets in that region. Outside the training range of a feature, the sample falls into the outermost leaf and receives that leaf's value—the prediction surface is flat beyond the data, and no ensemble of trees fixes it: sums and averages of range-bounded piecewise-constant functions are still range-bounded and piecewise-constant. Contrast a linear model, whose slope extends everywhere. Where it bites: **time series with trend** (a GBDT forecaster flatlines at roughly the training maximum while demand keeps growing—the canonical symptom), pricing/volume features drifting beyond history, and any covariate-shift regime. Fixes: transform the problem so the target is range-stationary—predict differences or growth rates, detrend first, or use a hybrid (linear/ETS trend plus GBDT on residuals); details in Chapter 7.

Red flags (what NOT to say):

- "Just add more trees / deeper trees"—capacity is irrelevant; the hypothesis class is range-bounded.
- Unable to name the time-series consequence, the context where this question is nearly always asked.
- Claiming trees can't handle trends at all (they can, *within* the training range—the failure is specifically beyond it).

What the interviewer is testing: Whether you can reason from the hypothesis class to a deployment failure and its standard fixes—a small question that reliably exposes memorized-vs-understood tree knowledge.

What separates good from great:

- L5 Piecewise-constant argument plus the flatlining-forecast example and one fix.
- L6 Multiple fixes with trade-offs (differencing changes the noise structure; hybrids

add moving parts) and the covariate-shift generalization beyond time series.

Common follow-ups: "Your GBM forecast flatlines at the recent maximum—walk me through the fix." "Does this affect classification too?" "How would you detect that you're in extrapolation territory at serving time?"

Interview Question

Q: Your random forest's impurity-based feature importance says `session_id_hash` is the top feature. The PM wants to build product strategy around the ranking. What do you tell them?

L6 Debugging ★★○

Strong answer:

Two separate alarms. **Alarm 1: the ranking is untrustworthy.** MDI is biased toward high-cardinality features—more candidate split points means more chances to cut impurity by luck—and it is computed on training data, so it rewards memorization. A hash of a near-unique ID at the top of an MDI list is close to a textbook artifact. **Alarm 2: it may not be an artifact—it may be leakage.** If splits on an ID-like feature genuinely predict the target out-of-sample, some target information is riding on it (group structure, temporal ordering encoded in ID assignment, duplicates across train/validation). So: recompute importance properly—permutation importance on a *held-out, group-aware* split (or drop-column retraining for the features that matter)—and audit `session_id_hash` for leakage before anything else. If permutation importance on clean held-out data sends it to zero, it was MDI cardinality bias; if it survives, chase the leak. And for the PM: *no* importance method supports causal product strategy—importance describes what the model uses, not what moves the outcome; if strategy hangs on it, that is an experiment (Chapter 6).

Red flags (what NOT to say):

- Taking the MDI ranking at face value, or "fixing" it by just deleting the feature without asking why it scored.
- Not knowing MDI's cardinality bias or that it is computed on training data.
- Endorsing a causal reading of any feature importance for strategy decisions.

What the interviewer is testing: Applied skepticism in the right order: distrust the estimator, then suspect the data, then guard the decision. Candidates who know all three importance pathologies but skip the leakage hypothesis miss the highest-stakes branch.

What separates good from great:

- L5 Names MDI's biases and proposes permutation importance on held-out data.
- L6 Adds the leakage branch with the group-aware split, drop-column as the arbiter, and the not-causal boundary for the PM conversation.
- L7 Sketches the full audit (feature-timestamp check, single-feature-dominance test, retrain-without and compare), and frames what *would* justify strategy: an experiment or causal analysis, with the model's importance as hypothesis generator only.

Common follow-ups: "Permutation importance also has a correlated-features problem—explain it." "How exactly could a session ID leak the target?" "What would you show the PM instead?"

Interview Question

Q: Design the serving story for a GBDT fraud model: p99 model latency under 2 ms at 20K QPS, with a compliance requirement that risk never decreases as chargeback count increases.

L7 System Design ★ ○ ○

Strong answer:

Decompose into model shape, execution, and the constraint. **Model shape**: inference cost $\approx T\times$ average depth; at equal accuracy prefer more, shallower trees (better vectorization and branch behavior), and treat (T, depth, η) as jointly tuned against the latency budget—train several accuracy-latency points and pick on the Pareto frontier. If a large model is materially better, distill: refit a smaller ensemble on the large model's scores. Consider CatBoost-style oblivious trees, whose lookup-table inference is the fastest of the tree family. **Execution**: never serve through a Python training API—compile (Treelite/lleaves-class native code or ONNX Runtime) for 2–10× single-row gains; batch scoring where the product allows; pin the model in memory (a few MB); and profile end to end, because at 20K QPS feature fetching, not the model, is usually the p99 driver—push feature retrieval to a low-latency store and precompute aggregates [DL 22]. **The constraint**: this is exactly a **monotonic constraint** on `chargeback_count`—enforced at split time by the library, guaranteed by construction, auditable to compliance—at a small, measurable accuracy cost. Close the loop with calibration (thresholded risk scores must mean what they say—Chapter 4) and a shadow-deployment latency test before cutover.

Red flags (what NOT to say):

- Reaching for "smaller model" without the compile-and-profile step, or ignoring that features dominate p99.
- Hand-rolling the monotonicity requirement (post-hoc score clamping, feature tricks) when every major library enforces it natively.
- No calibration story for a model whose scores gate money decisions.

What the interviewer is testing: Whether GBDT knowledge extends past training into the serving and governance realities where staff engineers actually spend time: latency decomposition, compilation, constraints as first-class tools.

What separates good from great:

- L5 Trees-times-depth cost model, compilation, monotonic constraints named.
- L6 The Pareto-frontier tuning framing, feature-fetch-dominates-p99 insight, and the split-time enforcement mechanism for monotonicity.
- L7 Distillation option, oblivious-tree trade-off, calibration and shadow-test governance, and a crisp accounting of which risks are model-side vs. platform-side.

Common follow-ups: "How do monotonic constraints actually restrict split finding?" "Where does the 2 ms actually go—break down a request." "The compliance team wants an explanation per declined transaction—what do you serve, at what cost?"

Chapter 3

Unsupervised Learning

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).

3.1 k-means

3.2 Gaussian Mixtures and EM

3.3 Hierarchical Clustering and DBSCAN

3.4 PCA

3.5 t-SNE and UMAP: Caveats First

3.6 Interview Questions

Chapter 4

Probabilistic Foundations

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).

4.1 MLE, MAP, and Full Bayes

4.2 The Bias–Variance Decomposition

4.3 Calibration and Proper Scoring Rules

4.4 Uncertainty: Bootstrap, Quantiles, Conformal

4.5 Interview Questions

Part II

Practice

Chapter 5

The Applied Craft

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).

5.1 Feature Engineering

5.2 The Leakage Taxonomy

5.3 Imbalanced Learning

5.4 Model Selection and Cross-Validation Failure Modes

5.5 Interpretability: SHAP and Its Traps

5.6 Interview Questions

Chapter 6

Experimentation and Causal Inference

TL;DR

This chapter is the program's canonical home for experimentation statistics. It starts where a first course stops: not "what is a p-value" but why your flat result is a power problem, how CUPED buys you a 2× sample-size saving for free, what to do when you must monitor a running test, and why a user-randomized test on a marketplace measures the wrong thing. It then covers uplift modeling (targeting persuadables, not the already-loyal), bandits (regret instead of inference), and the observational toolkit you reach for when randomization is impossible—each with the assumption that does all the work and the one failure mode that kills it. Ranking-specific experimentation (interleaving) lives in [SR 7]; LLM-evaluation statistics live in [DL 23]; both point here for the machinery.

6.1 Hypothesis Testing, Rebuilt

Every experiment answers one question: is the difference I measured bigger than the difference I would expect from noise alone? The null hypothesis H_0 says the treatment changed nothing; the test statistic measures the observed gap in units of its own standard error; the p-value is the probability of seeing a gap at least this extreme *if the null were true*. That conditional is the whole subject, and it is where interview answers go wrong.

Key Insight

What a p-value is not. It is not the probability that the null is true. It is not the probability your result was a fluke. It is not one minus the probability the treatment works, and $p = 0.04$ does not mean a 96% chance of a real effect—that quantity depends on the prior probability that your idea was any good, which at most companies is well under half. A p-value is a statement about data given a hypothesis, never about a hypothesis given data. The practical consequence: in an organization shipping mostly neutral ideas, a large share of "significant" wins at $p < 0.05$ are false, which is why replication and holdbacks exist.

For a binary metric—conversion, click, signup—the workhorse is the two-proportion z-test. With $\hat{p}_A, \hat{p}_B$ and sizes n_A, n_B, the statistic is the difference over its pooled standard error, $z = (\hat{p}_B - \hat{p}_A)/\sqrt{\hat{p}(1-\hat{p})(1/n_A + 1/n_B)}$ with $\hat{p}$ the pooled rate. For a continuous metric—revenue per user, session length, latency—use Welch's t-test, which does not assume equal

variances across arms; equal-variance Student's t buys nothing in practice and fails exactly when treatment changes the spread rather than the mean, which is a real and interesting outcome.

The central limit theorem is what makes this work on metrics that are nothing like normal. Revenue per user is a spike at zero plus a long right tail; the *mean* of millions of such draws is still approximately normal, so the test is valid at experiment scale even though the underlying data is grotesque. The caveats are practical, not theoretical: with heavy tails, convergence is slower and a handful of whales can dominate the variance, so cap or winsorize the metric at a pre-registered percentile (and report both capped and uncapped), analyze log or rank transforms as secondary readouts, or bootstrap when n is small and the tail is fat. Choosing the transform after seeing which one gives significance is p-hacking, so fix it in the plan.

Warning

One-sided tests. A one-sided test is legitimate only when a negative effect would lead to exactly the same decision as a null effect—rare, since most launches have guardrails you would honor if the metric moved down. Switching to one-sided after seeing the direction of the estimate is not a test; it is a 10% test wearing a 5% label. Default to two-sided.

6.2 Power and Sample Size

Power is the probability of detecting an effect that is genuinely there. Underpowered tests are the single largest source of wasted experiment capacity in industry: they produce a stream of ambiguous flat results, and—worse—the ones that do reach significance are systematically overestimated (Section 6.7).

Sample size per arm. For a two-sided test at level α with power $1-\beta$, detecting an absolute difference δ in a metric with per-unit variance σ^2:

$$n \approx \frac{2\sigma^2 \left(z_{1-\alpha/2} + z_{1-\beta}\right)^2}{\delta^2}.$$

At the conventional $\alpha = 0.05$, $1-\beta = 0.8$: $(1.96 + 0.84)^2 = 7.84$, so $n \approx 15.7\,\sigma^2/\delta^2$, universally quoted as

$$n \approx \frac{16\sigma^2}{\delta^2}, \qquad \text{and for a proportion } (\sigma^2 = p(1-p)): \quad n \approx \frac{16\,p(1-p)}{\delta^2}.$$

Three consequences worth internalizing: sample size scales with the *square* of the inverse effect size (halving the detectable effect costs 4× the traffic); it is linear in variance (hence Section 6.3); and α and β enter only through a constant.

Work the canonical case. A 2% baseline click rate, and you want to detect a 1% *relative* lift—that is $\delta = 0.02 \times 0.01 = 0.0002$ absolute. Then $n \approx 16(0.02)(0.98)/(0.0002)^2 = 0.3136/4 \times 10^{-8} \approx 7.84$ million users per group, 15.7M total. If the product sees 2M eligible users per day, that is about eight days—and only if every user is exposed. This arithmetic is why "we'll just run it for a week" is not a plan and why 1%-relative effects are undetectable at most companies: the honest answer is often "we cannot measure this, so decide it on other grounds or bundle it into a bigger change."

Run the formula backwards more often than forwards. Fix the traffic you actually have and solve for δ: that is the minimum detectable effect (MDE), and it belongs in the design doc before anyone builds anything. If the MDE is 3% and no one believes the feature moves the metric by more than 1%, the experiment is decoration. Note also that MDE is a threshold for *detection*, not the effect you should expect to see.

Duration is not just n/traffic. Weekly seasonality means an experiment must cover whole weeks (weekday and weekend populations differ in composition and behavior); a Tuesday-to-Friday test measures Tuesday-to-Friday users. Learning and novelty effects (Section 6.7) argue for at least one full cycle, typically 1–2 weeks minimum regardless of what power says. Cookie churn and multi-device users erode assignment stability in longer tests, which argues against running for months. And the unit of analysis must match the unit of randomization—see the delta method in Section 6.6.

6.3 Variance Reduction

Since $n \propto \sigma^2$, halving variance halves the traffic bill—or halves the MDE-squared at fixed traffic. This is the highest-leverage statistical work in an experimentation platform, and CUPED is the standard tool.

CUPED (Controlled-experiment Using Pre-Experiment Data). Let Y be the metric and X a pre-experiment covariate—almost always the *same metric measured before the experiment*—which is unaffected by treatment. Define the adjusted metric

$$Y' = Y - \theta\,(X - \bar{X}), \qquad \theta^\star = \frac{\mathrm{Cov}(Y, X)}{\mathrm{Var}(X)}.$$

Y' is unbiased for the same treatment effect ($\mathbb{E}[Y'] = \mathbb{E}[Y]$ because $\mathbb{E}[X - \bar{X}] = 0$), and at $\theta^\star$ its variance is

$$\mathrm{Var}(Y') = \mathrm{Var}(Y)\,(1 - \rho^2), \qquad \rho = \mathrm{Corr}(Y, X).$$

The mechanism is worth stating in words, because that is what an interviewer wants: users differ enormously in their baseline activity, and that between-user heterogeneity is noise with respect to the treatment effect. CUPED subtracts off the part of each user's outcome that was predictable from their own past, leaving a smaller residual to compare across arms. It is ordinary regression adjustment, chosen so the adjustment cannot itself be affected by treatment.

The payoff is concrete. With $\rho = 0.7$—typical when the covariate is the same metric over the prior two weeks—variance drops by $1 - 0.49 = 0.51$, so you need about 51% of the sample, close to a $2\times$ saving; the 7.84M-per-group test above becomes roughly 4M per group. With $\rho = 0.5$ the saving is 25%. With $\rho = 0.3$, 9%—real but not worth much complexity.

CUPED fails, or does nothing, in identifiable situations: new users have no pre-period (run CUPED on the returning-user stratum and report the new-user stratum separately, or use a covariate available at signup); metrics with weak self-correlation, such as rare conversions, give small ρ; and any covariate contaminated by treatment—measured after assignment, or a post-treatment segment—reintroduces bias, which is the one genuine danger. Estimating θ on the pooled experiment data is standard and its bias is $O(1/n)$, negligible at experiment scale.

Two simpler cousins are worth knowing. *Stratification* (assign within strata, e.g., country $\times$ platform $\times$ activity decile) removes the same between-group variance by design rather than by

adjustment; *post-stratification* does it in analysis. And CUPED generalizes: replace the single covariate with a regression on many pre-period features, or a model prediction of Y—variance reduction is then governed by the model's R^2. The intuition is unchanged: any variance you can explain without touching treatment is variance you do not have to pay for.

6.4 Sequential Testing and the Cost of Peeking

The fixed-horizon test is a promise: choose n in advance, look once. Everyone breaks it, because dashboards update hourly and a bad launch should be killed today, not next Thursday.

Warning

Quantify the peeking problem. Under a true null, a fixed-horizon test crosses $p < 0.05$ at some point with probability far above 5% if you keep looking: roughly 14% with 5 equally spaced looks and around 20–25% with daily looks over two weeks, rising toward 100% as monitoring becomes continuous and n grows without bound. "We only shipped it because it hit significance" is, under repeated peeking, a statement about your stopping rule rather than about your feature.

Three legitimate responses. *Group-sequential* designs (alpha spending) pre-specify K interim analyses and split α across them: O'Brien–Fleming spends almost nothing early—so early stopping requires an overwhelming effect—and nearly the full α at the final look, which keeps final-analysis power close to the fixed-horizon design; Pocock spends evenly, stopping earlier for smaller effects at the cost of final power. *Always-valid inference* (mixture SPRT, confidence sequences) gives p-values and intervals valid at *every* sample size, so you may stop whenever you like; the price is width—an always-valid interval is materially wider than a fixed-horizon one at the same n, i.e., you pay in sample size for the right to look continuously. *Bayesian monitoring* with a decision rule on posterior quantities is defensible when the prior and loss are honestly specified, but is not a loophole around the multiplicity it looks like it evades.

The pragmatic answer for most organizations, and the one to give in an interview: fix the horizon and pre-register the duration for the launch decision, monitor *guardrails only* for harm with an explicit stopping rule (large regressions warrant killing a test on a much stricter threshold, and stopping early for harm does not inflate false wins), and adopt always-valid inference platform-wide if teams genuinely need to act on interim readouts. What is not defensible is the unstated hybrid: monitoring everything daily, stopping on the first green, and reporting a fixed-horizon p-value.

6.5 Multiple Testing

Every experiment reads out dozens of metrics across dozens of segments; at $\alpha = 0.05$, twenty independent null comparisons produce one "significant" result on average. Two different error rates are worth controlling, and the choice is about consequences.

Family-wise error rate (FWER) is the probability of *any* false positive; control it when a single false alarm is expensive—the primary metric, and the guardrail set that gates the launch. Bonferroni (α/m) is crude and conservative but unimpeachable and fine for the handful of decision metrics. False discovery rate (FDR) is the expected *fraction* of false positives among rejections; control it with Benjamini–Hochberg when you are scanning many exploratory metrics

or segments and can tolerate some false leads, because BH is far more powerful than Bonferroni at scale.

Warning

Segment fishing. Slicing a flat experiment by country × platform × tenure × acquisition channel generates hundreds of comparisons; some will be "significant," and the story you tell about the winner will be plausible. Pre-register the segments that matter, apply FDR to exploratory slices, and treat anything surviving only in one unregistered cell as a hypothesis for the next experiment—never as a finding, and never as a reason to ship to that segment.

6.6 Metric Design and the Randomization Unit

The OEC (overall evaluation criterion) is the metric the experiment is allowed to be judged on, chosen in advance. It should be sensitive enough to move within a reasonable horizon, aligned with long-term value, and hard to game. The politics are the hard part: a single OEC forces the trade-offs into the open (engagement versus revenue, growth versus quality), which is why organizations avoid choosing one and end up shipping on whichever metric happened to move. Guardrail metrics—latency, crash rate, unsubscribe, support contacts, revenue—exist so a win on the OEC cannot silently break the business, and they are evaluated as non-inferiority checks, not as things to be maximized.

Ratio metrics are where careful candidates separate themselves. Click-through rate is $\sum \text{clicks} / \sum \text{impressions}$, but randomization is by user; the denominator is itself random and correlated with the numerator, so the naive standard error computed over events is wrong—usually too small, sometimes badly, because it ignores within-user correlation and treats every event as independent.

Delta method for a ratio. For $R = \bar{X}/\bar{Y}$ with per-user totals X_i (clicks) and Y_i (impressions),

$$\operatorname{Var}(R) \;\approx\; \frac{1}{\mu_Y^2}\operatorname{Var}(\bar{X}) \;-\; \frac{2\mu_X}{\mu_Y^3}\operatorname{Cov}(\bar{X},\bar{Y}) \;+\; \frac{\mu_X^2}{\mu_Y^4}\operatorname{Var}(\bar{Y}),$$

with all moments taken over the *randomization unit*. Equivalently, bootstrap over users, or run the test on the per-user linearized metric $L_i = (X_i - R\,Y_i)/\mu_Y$—all three agree and all three respect the clustering.

The general rule: **analyze at the unit you randomized**. If you randomize users and analyze sessions, your effective sample size is inflated by the number of sessions per user and your p-values are optimistic; the fix is clustered standard errors, the delta method, or a per-user metric.

Interference breaks the deeper assumption (SUTVA): that one unit's outcome does not depend on another unit's assignment. On a two-sided marketplace, treatment users who book more leave less supply for control users, so the measured gap overstates the true effect—a pure transfer can look like growth. In social products, treatment users change the experience of their control-group friends, diluting the measured effect toward zero. The designs that address it: *cluster randomization* (randomize by region, network community, or supply pool; unbiased, but the effective sample size becomes the number of clusters, which is a brutal variance cost),

switchback (randomize region × time bucket, alternating treatment on and off; well-suited to marketplace pricing/dispatch, with carryover bias handled by burn-in periods within each bucket), and *budget-split* designs in ads. The interview signal is not knowing the names—it is spotting that a user-randomized marketplace test answers the wrong question at all.

6.7 The Pathologies That Decide Interviews

Key Insight

SRM is the first thing you check. Sample ratio mismatch means the observed split differs from the intended one—50/50 designed, 50.4/49.6 observed. A chi-square goodness-of-fit test against the intended ratio, flagged at $p < 0.001$, catches it. At experiment scale even a 0.2% deviation is wildly improbable by chance, so an SRM means the assignment or logging pipeline is broken: bot filtering applied asymmetrically, redirect latency dropping treatment users, a crash in one arm removing its worst-affected users, triggering logic evaluated differently across arms, or dedup joining on a field one arm populates differently. **An experiment with an SRM is not analyzable**—the arms are no longer comparable populations, and the very users who dropped are usually the ones most affected. Debug it; never "adjust for" it.

Novelty and primacy effects: users react to change itself. A new UI draws exploratory clicks that decay (novelty), or long-tenured users are temporarily slower under a changed layout and recover (primacy). Both mean the week-one effect is not the steady-state effect. Detect them by plotting effect-by-exposure-day and by comparing new versus existing users; measure the steady state with a longer test or a holdback (Section 6.12).

Twyman's law: any figure that looks interesting or unusual is usually wrong. A +30% revenue result is a logging bug, a bot, or a whale until proven otherwise—check SRM, check the distribution and the top percentile, check whether the metric definition changed, and only then celebrate. In practice the largest observed effects are dominated by instrumentation errors, because the true distribution of effect sizes is concentrated near zero.

The winner's curse: conditional on being significant, an estimate is biased upward—selection on the noisy estimate favors draws where noise helped. The bias is severe exactly when power is low: with 20% power, a significant estimate can overstate the truth several-fold, which is the arithmetic reason underpowered experiment programs report launch effects that never appear in the annual numbers. Remedies: raise power, apply shrinkage (empirical-Bayes toward the prior mean of effects, which at most companies is near zero), and validate the biggest wins with a replication or a holdback.

Finally, A/A tests and pre-experiment bias checks: run the analysis pipeline on two identical arms and confirm the p-value distribution is uniform and the false-positive rate is nominal. A/A failures reveal exactly the bugs that make A/B results untrustworthy—clustered units treated as independent, ratio metrics with wrong standard errors, assignment leakage—and they are the cheapest audit an experimentation platform can run continuously.

6.8 Heterogeneous Treatment Effects

The average treatment effect hides that the same change may help one group and harm another. Legitimate HTE analysis is pre-registered ("we expect the effect to be larger on mobile, where

the old flow was worst"), corrected for multiplicity, and treated as directional unless replicated. Illegitimate HTE analysis is the segment fishing of Section 6.5 with a machine-learning veneer.

Modern approaches estimate the conditional average treatment effect $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid X = x]$ directly—causal forests, DR-learners, and the meta-learners of Section 6.9—and then validate the *ranking* they induce on held-out randomized data rather than reporting per-segment p-values. That reframing is what makes HTE actionable: you rarely need the effect for a segment, you need to know whom to target, which is precisely uplift modeling.

6.9 Uplift Modeling

The motivating story is a retention campaign. You have a churn model, so you send the discount to the users most likely to churn—and the campaign loses money. The reason is that churn probability is not treatment effect.

Key Insight

The four quadrants. Every user is one of: *persuadables* (convert only if treated—the only group worth spending on), *sure things* (convert either way—the discount is pure margin loss), *lost causes* (convert neither way—wasted spend), and *sleeping dogs* (treatment makes them *worse*: the reactivation email reminds a dormant user to cancel). Targeting by outcome probability buys sure things and wakes sleeping dogs. Targeting by *uplift*—the individual treatment effect $\tau(x)$—buys persuadables. This is the whole subject in one paragraph, and it is the answer to the most common interview question in the area.

Meta-learners turn any supervised model into a CATE estimator. The *S-learner* fits one model on the pooled data with treatment as a feature and predicts $\hat{\tau}(x) = f(x, 1) - f(x, 0)$; simple, but any regularization that shrinks the treatment coefficient shrinks the estimated effect toward zero, which is fatal when effects are small relative to outcome signal. The *T-learner* fits separate models on treated and control and differences them; it avoids that bias but differencing two independently noisy models amplifies variance, and it wastes the shared structure. The *X-learner* imputes individual effects using the cross-model predictions, fits models on those imputed effects, and combines them weighted by the propensity—designed for badly imbalanced treatment groups, which is the common industry case (a 5% campaign holdout). Doubly-robust learners (DR-learner) and *causal forests*, which split on effect heterogeneity rather than outcome and give honest confidence intervals via sample splitting, are the current defaults when data volume allows.

Evaluation is the part people get wrong, because ground-truth individual effects do not exist. Outcome AUC validates nothing about uplift—a perfect churn model can have zero targeting value. Instead, on *randomized* held-out data, sort users by predicted uplift and compare the observed treatment-versus-control outcome gap in the top $k\%$ against the overall gap: that is the uplift curve, and its cumulative form (incremental conversions gained versus the random-targeting diagonal) is the Qini curve, summarized by the area between them. Randomized holdout data is not optional here; without it you cannot observe the counterfactual gap at all.

6.10 Bandits

An A/B test is designed for *inference*: a clean, unbiased estimate of an effect. A bandit is designed for *regret minimization*: cumulative reward, allocating traffic toward arms that look

good while retaining enough exploration to notice when they are not. Regret is the objective, $R_T = \sum_t (\mu^\star - \mu_{a_t})$.

ϵ-greedy plays the best-known arm with probability $1-\epsilon$ and a random arm otherwise—trivial to implement, but a fixed ϵ never stops exploring, giving linear regret; decaying $\epsilon_t \propto 1/t$ fixes the asymptotics. *UCB* is optimism under uncertainty: play $\arg\max_a \hat{\mu}_a + \sqrt{2 \ln t / n_a}$, where the bonus is a confidence radius that shrinks as an arm is pulled, so arms are abandoned only once their uncertainty is small enough to rule them out. *Thompson sampling* is posterior sampling: maintain a posterior per arm, draw one sample from each, play the argmax. For Bernoulli rewards with a Beta prior the mechanics are three lines—arm a has $\text{Beta}(\alpha_a + s_a, \beta_a + f_a)$ after s_a successes and f_a failures; sample θ_a from each; play the max; update—and its exploration is automatically proportional to the probability that an arm is optimal. Thompson sampling generally matches or beats UCB empirically, handles delayed and batched feedback gracefully (the posterior simply lags), and needs no tuning constant, which is why it is the production default.

Bandits win when there are many arms and little traffic per arm, when content is perishable (news, promotions, creatives) so that a two-week test outlives the item, when the reward is cheap and immediate, and when regret genuinely matters because traffic sent to bad arms is money lost. A/B wins when you need an unbiased, precisely quantified effect with guardrails—a launch decision, a pricing change, anything with a legal or financial commitment—because adaptive allocation makes naive post-hoc analysis invalid (the data is no longer i.i.d. across arms). Rewards that arrive days later, strong non-stationarity, and the need to explain a single number to a review board all favor the fixed test.

Contextual bandits condition the arm choice on features: LinUCB models each arm's reward as linear in the context with a ridge estimate and adds a confidence bonus from the covariance, giving a per-context optimism rule. The essential companion is off-policy evaluation: because you logged the propensity $p(a \mid x)$ under the deployed policy, you can estimate a new policy's value from logs by importance weighting, $\hat{V}_{\text{IPS}} = \frac{1}{n} \sum_i \frac{\mathbb{1}[a_i = \pi(x_i)]}{p(a_i \mid x_i)} r_i$—unbiased when every action had nonzero logging probability, but high-variance when the new policy diverges from the logged one, which is why you clip weights, use self-normalized or doubly-robust variants, and *log propensities from day one*. Practical failure modes to name: delayed feedback (update with partial rewards or model the delay), non-stationarity (discount old data or use sliding windows—a bandit that has converged is blind to a changed world), and batch updates (real systems update hourly, not per-request, which slows learning and must be simulated before launch).

6.11 Observational Causal Inference

Sometimes randomization is impossible: the feature shipped to everyone, prices cannot be varied by user for fairness and legal reasons, or the question is retrospective. The potential-outcomes framework makes the target explicit: each unit has $Y(1)$ and $Y(0)$, only one is observed (the fundamental problem of causal inference), and we estimate averages—ATE over everyone, ATT over the treated, CATE within covariate strata.

Key Insight

Three assumptions do all the work. *Ignorability* (no unmeasured confounding): treatment is as-good-as-random given the observed covariates—untestable, always the weak link, and the honest headline of any observational analysis. *Positivity/overlap*: every co-

variate profile has a nonzero chance of both treatments, or you are extrapolating rather than comparing. *SUTVA*: no interference and one version of treatment. Method choice is really the choice of which assumption you find least implausible in this dataset.

Propensity scores and IPW. The propensity $e(x) = P(T = 1 \mid X = x)$ summarizes confounding in one number; weighting each unit by $1/e(x)$ (treated) and $1/(1 - e(x))$ (control) reconstructs a pseudo-population where treatment is independent of X. The failure mode is arithmetic and severe: units with propensities near 0 or 1 receive enormous weights, so a handful of observations dominate the estimate and variance explodes. Trim extreme propensities or use stabilized weights, always inspect the overlap of the propensity distributions across arms (non-overlapping regions are not estimable and should be excluded and reported), and check covariate balance after weighting via standardized mean differences rather than by re-running significance tests. Doubly-robust (AIPW) combines an outcome model with the propensity model and is consistent if *either* is correctly specified—worth naming, and worth being honest that it does not rescue you from unmeasured confounding.

Difference-in-differences. With a treatment that turned on for one group at a known time, compare the change over time in the treated group to the change in an untreated group: $\hat{\tau} = (\bar{Y}_T^{\text{post}} - \bar{Y}_T^{\text{pre}}) - (\bar{Y}_C^{\text{post}} - \bar{Y}_C^{\text{pre}})$, which differences away both fixed group differences and common time shocks. Everything rests on *parallel trends*: absent treatment, the two groups would have moved together. Probe it by plotting pre-period trends and running an event-study specification with leads and lags—a pre-trend that diverges before treatment falsifies the design. A concrete failure: a feature rolled out to the most engaged market first, whose growth was already accelerating; DiD attributes the acceleration to the feature. Staggered adoption across many units needs the modern estimators rather than naive two-way fixed effects, which is worth one sentence of awareness.

Instrumental variables. An instrument Z moves treatment but affects the outcome *only* through treatment (relevance plus the exclusion restriction), so the Wald estimator $\hat{\tau} = \frac{\text{Cov}(Y,Z)}{\text{Cov}(T,Z)}$ (2SLS in general) recovers a causal effect despite unmeasured confounding. The industry-realistic version is an encouragement design: randomize an *invitation* to use a feature you cannot force people to adopt, then use assignment as the instrument for actual usage. Caveats to state unprompted: weak instruments (low first-stage correlation) produce badly biased and unstable estimates—report the first-stage F; the exclusion restriction is untestable and usually the point of attack (does the invitation itself change behavior, apart from adoption?); and IV identifies a LATE, the effect for *compliers* only, which may not be the population you care about.

Regression discontinuity. Where treatment is assigned by a threshold on a running variable—a credit score cutoff, a pricing tier boundary, a rank-based promotion—units just above and just below are comparable, so a local comparison at the cutoff is nearly experimental. Sharp RD assigns deterministically at the threshold, fuzzy RD only shifts probability (and is then an IV). It is credible but local (the effect at the cutoff), sensitive to bandwidth choice, and invalidated by manipulation of the running variable, which is checked with a density test at the threshold.

Warning

The decision guide. Experiment if you can—including on a small slice, in one market, or via an encouragement design, all of which beat any observational estimate. If you cannot: a threshold rule in the assignment mechanism points to RD; a staggered rollout across units points to DiD; a randomized nudge you can exploit points to IV; rich covariates plus

a plausible selection story and good overlap points to propensity/DR methods. State the identifying assumption out loud, run the diagnostic that could falsify it, and report the estimate with that caveat attached. And never read a causal effect off a predictive model's feature importances or SHAP values—those describe the model, not the world.

6.12 Experimentation at Staff Level

Individual tests are the easy part; the staff-level question is whether the organization's experiments add up. Experiment review—a lightweight design check before launch (OEC, MDE, duration, guardrails, segments pre-registered) and a results check before shipping—catches most of the pathologies in this chapter at the point where they are still cheap to fix. Institutional memory matters as much: a searchable archive of past experiments, including the failures, prevents the same idea from being re-run every eighteen months and enables meta-analysis of the effect-size distribution, which in turn calibrates priors, shrinkage, and how much to believe the next surprising win.

The recurring hard call is shipping on flat. A flat result is not evidence of no effect; it is evidence that the effect is smaller than your MDE. The decision then rests on asymmetric costs: if the change reduces technical debt, unblocks future work, or is strategically necessary, ship it on flat and say so explicitly; if it adds complexity or risk with no measured payoff, do not. Long-run effects deserve separate machinery—holdback groups (a small population kept on the old experience for months) measure the accumulated effect of many launches and routinely reveal that the sum of individually significant wins is smaller than their arithmetic total.

6.13 Interview Questions

Interview Question

Q: We want to detect a 1% relative lift on a 2% click-through rate. How many users do we need, and how long should we run?

L5 Estimation ★★★

Strong answer:

Convert to an absolute effect first: 1% relative on a 2% base is $\delta = 0.0002$. Use $n \approx 16\,p(1-p)/\delta^2$ at 80% power and $\alpha = 0.05$ (the 16 is 2×2.8^2, from $z_{0.975}+z_{0.8} = 1.96+0.84$). That gives $n \approx 16(0.02)(0.98)/4 \times 10^{-8} \approx 7.84$M per group, about 15.7M users total. At 2M eligible users per day that is roughly eight days of exposure, and you round up to two full weeks so the test covers whole weekly cycles rather than a weekday-skewed population. Then say the things that separate levels. The honest framing is MDE, not sample size: with the traffic we have, what is the smallest effect we can see? If the answer is larger than any plausible effect, do not run the test. Variance reduction changes the arithmetic materially—CUPED on the pre-period click rate with $\rho \approx 0.7$ cuts required n roughly in half. Only users who actually reach the surface count, so if the feature triggers for 30% of traffic, the exposure-based denominator is what matters and the calendar time triples. And CTR is a ratio metric: if we randomize by user, compute the variance by the delta method or bootstrap over users rather than over impressions, or the standard error will be too small.

Red flags (what NOT to say):

- Applying the relative lift as if it were absolute ($\delta = 0.01$), which understates the requirement by four orders of magnitude.
- Quoting a sample-size number with no α, power, or variance assumption attached.
- Ignoring the trigger/exposure population and computing duration off total site traffic.
- Treating impressions as independent samples for a user-randomized test.

What the interviewer is testing: Whether experiment sizing is a reflex or a formula you half-remember. The relative-to-absolute conversion and the square dependence on δ are where most candidates fail; the strong ones volunteer MDE and CUPED without being asked.

What separates good from great:

- L5 Correct arithmetic with stated assumptions.
- L6 Reframes as MDE, adds CUPED and the exposure denominator, handles the ratio-metric variance.
- L7 Questions whether a 1% CTR lift is the right thing to measure at all, and proposes bundling small changes or measuring a downstream metric with better signal-to-noise.

Common follow-ups: "How does the answer change at 95% power?" (the constant moves from 15.7 to about 26). "What if the metric is revenue per user instead?" (use the observed variance, expect heavy tails, cap or bootstrap.)

Interview Question

Q: Your test came back flat, but you are confident the feature helps. What do you do?

L6 Trade-off ★★★

Strong answer:

Start by refusing the framing: flat is not "no effect," it is "no effect larger than my MDE," so the first move is to compute the confidence interval and ask what it excludes. A CI of $[-0.5\%, +0.7\%]$ is uninformative about a 0.3% effect; a CI of $[-0.05\%, +0.06\%]$ genuinely rules out anything worth shipping for. That distinction determines everything that follows. If the test was underpowered, buy power rather than looking harder: CUPED or stratification on pre-period behavior (a $2\times$ effective saving at $\rho = 0.7$), restrict to the triggered population so untouched users stop diluting the effect, choose a more sensitive metric closer to the mechanism (the specific click the feature changes, not sitewide revenue), extend duration in whole weeks, or increase allocation. If power is adequate and the interval is tight, take the result seriously: check whether the feature actually shipped correctly (instrumentation, exposure rates, an SRM check), then look at pre-registered segments and the effect-by-day curve for novelty or dilution masking a real subgroup effect—and treat any unregistered segment win as a hypothesis for a follow-up test, not a finding. Sequential or always-valid monitoring is the answer to "can I keep looking," not a way to rescue this result.

Close on the decision, because that is what the interviewer wants: if the change is cheap, reduces complexity, or is strategically required, ship on flat and say explicitly that we are

shipping without measured benefit; if it adds risk or maintenance cost, do not ship. Either way, record the MDE so the next team knows what this experiment could and could not have detected.

Red flags (what NOT to say):

- Peeking until it turns significant, or slicing segments until one is.
- Treating $p > 0.05$ as proof of no effect.
- Extending the test indefinitely with no pre-specified stopping rule.
- Shipping anyway while claiming the experiment supported it.

What the interviewer is testing: Whether you distinguish absence of evidence from evidence of absence, and whether your instinct under pressure is to buy power or to fish for significance.

What separates good from great:

- L5 Notes the test may be underpowered and suggests running longer.
- L6 Uses the CI to decide, applies variance reduction and trigger-based analysis, keeps segment analysis disciplined.
- L7 Turns it into a program question—our MDEs are too large for the effects we ship, so invest in variance reduction and metric sensitivity rather than re-running this test.

Common follow-ups: "How would you decide whether to ship on flat?" "What would you tell a PM who wants the win?"

Interview Question

Q: Your treatment shows +2% on the primary metric, but the SRM check fired. What now?

L6 Debugging ★★★

Strong answer:

Stop and discard the result—an SRM means the arms are no longer comparable populations, so the +2% is uninterpretable, not merely noisy. Say that first and without hedging; the instinct to "adjust" or to ship because the number is good is the failure mode being tested.

Then debug systematically, from assignment to analysis. Confirm the mismatch is real (chi-square against the intended ratio, $p < 0.001$; at scale a 0.2% deviation is not chance). Check the assignment layer for a bucketing bug, an off-by-one hash range, or overlapping experiment layers. Check exposure logging: latency or crashes in one arm silently drop users, and a redirect that adds 200ms to treatment removes exactly the impatient users. Check triggering logic—if the trigger condition is evaluated after a code path that differs across arms, the populations diverge by construction. Check downstream filters: bot removal, dedup joins, and outlier exclusions applied asymmetrically are extremely common causes. Segment the ratio by day, platform, country, and app version to localize where the imbalance appears, which usually names the bug.

Note why the direction of the bias is unpredictable and often self-serving: if treatment lost its slowest, least engaged users to a crash, treatment's average looks better for a reason unrelated to the feature. After the fix, re-run—and consider an A/A test on the same

pipeline to confirm the fix took.

Red flags (what NOT to say):

- Reweighting the arms to "correct" the imbalance and reporting the effect anyway.
- Dismissing a 50.4/49.6 split as too small to matter at 10M users.
- Shipping because the effect is large and the SRM is "probably logging."
- No systematic hypothesis order—just re-running the query.

What the interviewer is testing: Whether you know SRM invalidates rather than perturbs an experiment, and whether you can generate a concrete, ordered list of pipeline causes.

What separates good from great:

- L5 Recognizes SRM as a red flag and escalates.
- L6 Declares the result unusable and runs the assignment → exposure → trigger → filter differential.
- L7 Fixes the class of bug—automated SRM alerting on every experiment, A/A coverage, and a policy that blocks readouts on SRM.

Common follow-ups: "What threshold would you alert on?" "How does an SRM in the trigger step differ from one in assignment?"

Interview Question

Q: Explain CUPED to a PM, and quantify what it buys us.

L6 Conceptual ★★○

Strong answer:

The PM version: our users differ wildly—some visit daily, some monthly—and most of the variation in any metric is that pre-existing difference, not anything our feature did. CUPED subtracts each user's own historical baseline before comparing groups, so we are measuring the change in behavior rather than the level. It cannot bias the result, because the baseline was measured before the experiment started and cannot be affected by treatment.

The quantified version: adjust $Y' = Y - \theta(X - \bar{X})$ with $\theta^\star = \mathrm{Cov}(Y, X)/\mathrm{Var}(X)$, which leaves the expectation unchanged and multiplies variance by $1 - \rho^2$. Since required sample size is proportional to variance, at $\rho = 0.7$ we need about 51% of the users—a two-week test becomes one week, or the same two weeks detect an effect $1/\sqrt{2}$ as large. At $\rho = 0.5$ the saving is 25%; below $\rho \approx 0.3$ it is not worth the plumbing.

Then the honest limits: it does nothing for new users (no pre-period), it is weak for rare conversions where self-correlation is low, and the covariate must be strictly pre-treatment—using a post-assignment covariate introduces real bias, which is the one way to get this wrong. Mention that stratified assignment achieves the same variance reduction by design, and that the idea generalizes to a regression on many pre-period features.

Red flags (what NOT to say):

- Claiming CUPED increases power "by reducing bias" or that it changes the estimand.
- Using a post-treatment covariate, or the metric itself during the experiment window.
- Quoting a fixed speedup with no reference to ρ.

What the interviewer is testing: Whether you can translate a variance-reduction technique into business terms and back, and whether you know its failure conditions rather than just its formula.

What separates good from great:

- L5 Describes it as "controlling for pre-period behavior."
- L6 Gives $\theta^\star$, the $1-\rho^2$ result, and the sample-size consequence with numbers.
- L7 Frames it as platform infrastructure—compute covariates once for all experiments—and connects it to stratification and regression adjustment as one family.

Common follow-ups: "What if θ is estimated on the experiment data itself?" "How would you handle new users?"

Interview Question

Q: Your marketplace A/B test shows +2% GMV in treatment. Why might the true effect be smaller, or negative?

L6 Conceptual ★★★

Strong answer:

Because user-level randomization violates SUTVA on a two-sided platform. Treatment and control users compete for the same finite supply: if the treatment ranking converts better, treated users book the scarce inventory first and control users find less of it, so control is actively depressed by the experiment. The measured gap is then part real effect and part transfer, and in the extreme—a pure reallocation with no new demand—the true incremental effect is zero while the test reads +2%. Related mechanisms: sellers reacting to changed demand, budget-constrained advertisers, and shared ranking models retrained on treatment-influenced data.

The designs that fix it: cluster randomization at the level where the market clears (city, region, supply pool), which is unbiased but reduces the effective sample size to the number of clusters and therefore costs a great deal of variance; switchback, randomizing region $\times$ time bucket and alternating the treatment on and off, which suits dispatch and pricing decisions and needs burn-in within each bucket to handle carryover; and budget-split designs in ads. A cheap diagnostic before any of that: vary the treatment allocation (say 10% versus 50%) and see whether the measured effect changes—a treatment-share-dependent effect is direct evidence of interference. Also check supply-side and marketplace-wide guardrails rather than only the treatment arm's GMV.

And run the ordinary checks too, since they explain many +2% results: SRM first, then novelty effects and whale concentration in the metric.

Red flags (what NOT to say):

- Accepting the +2% at face value with no interference discussion.
- Proposing user-level randomization with more users as the fix.
- Naming switchback without knowing what is randomized or why burn-in exists.

What the interviewer is testing: Whether you notice that the standard experimental assumption fails in two-sided markets—the single most-tested causal-inference idea at platform companies.

What separates good from great:

- L5 Mentions novelty and seasonality but misses interference.
- L6 Diagnoses cannibalization, proposes cluster or switchback with the variance trade-off stated.
- L7 Adds the allocation-varying diagnostic and reasons about which market-level metric would actually settle the question.

Common follow-ups: "How would you choose the cluster granularity?" "What breaks in a switchback if effects persist across buckets?"

Interview Question

Q: Why not send retention offers to the users with the highest churn probability?

L6 Conceptual ★★○

Strong answer:

Because churn probability is not treatment effect. Sorting by churn risk buys three kinds of users you do not want—sure things who would have stayed anyway (pure margin loss), lost causes who leave regardless (wasted spend), and sleeping dogs whom the offer actively reminds to cancel—and only incidentally the persuadables, the users whose behavior the offer actually changes. What you want to rank by is uplift $\tau(x) = \mathbb{E}[Y(1) - Y(0) \mid x]$.

To build it: run a randomized campaign with a proper holdout, then estimate CATE. A T-learner (separate retention models on treated and untreated) is the obvious start, an X-learner is better when the treated group is small—which it usually is with a 5% holdout—and a causal forest or DR-learner is the modern default with enough data. Validate on randomized held-out data with an uplift/Qini curve: sort by predicted uplift, and check that the observed treatment-minus-control gap in the top decile substantially exceeds the overall gap. Outcome AUC is not a validation of an uplift model and saying so is a differentiator.

Then the economics: target down the uplift ranking until incremental margin equals offer cost, and explicitly exclude negative-uplift users. Mention that this requires ongoing randomized holdouts—the moment you target purely by model score, you lose the data that lets you re-estimate the effect.

Red flags (what NOT to say):

- Treating a churn model as a targeting model with no notion of counterfactuals.
- Evaluating the uplift model with outcome AUC or accuracy.
- Training on observational campaign data where treatment was already targeted, without addressing confounding.
- Never mentioning that some users are harmed by the intervention.

What the interviewer is testing: Whether you can separate prediction from causation in a setting where a predictive model is right there and superficially plausible.

What separates good from great:

- L5 Says "we should target users the offer will change" without method.
- L6 Gives the four quadrants, a meta-learner choice with justification, and Qini evaluation on randomized data.

- L7 Adds the cost-based targeting threshold and the standing-holdout design that keeps the system measurable over time.

Common follow-ups: "Which meta-learner for a 5% treated group, and why?" "How do you detect sleeping dogs?"

Interview Question

Q: Thompson sampling, UCB, or ϵ-greedy—mechanics, and when would you use a bandit instead of an A/B test?

L6 Trade-off ★★○

Strong answer:

Mechanics, briefly. ϵ-greedy: exploit the empirical best, explore uniformly with probability ϵ; trivial, but constant ϵ gives linear regret and explores arms already known to be bad. UCB: play $\arg\max \hat{\mu}_a + \sqrt{2 \ln t / n_a}$—optimism in the face of uncertainty, where the bonus shrinks with pulls so an arm is dropped only once its uncertainty is small. Thompson sampling: keep a posterior per arm, draw one sample from each, play the argmax; for Bernoulli rewards with Beta priors that is Beta$(\alpha + s_a, \beta + f_a)$, sample, take the max, update. TS explores each arm in proportion to its probability of being optimal, needs no tuning constant, and degrades gracefully under delayed or batched feedback because the posterior simply lags—which is why it is the production default.

When a bandit: many arms with thin traffic, perishable content (creatives, headlines, promotions) whose value expires before a fixed test would finish, cheap immediate rewards, and a genuine cost to sending traffic to losers. When A/B: you need an unbiased, precisely quantified effect with confidence intervals for a launch or pricing decision; you have guardrails that must be evaluated on comparable populations; rewards are delayed by days; or the result must be legible to people who will not accept "the allocation adapted."

The subtle point worth making: adaptive allocation breaks naive post-hoc inference, so if you need both selection and estimation, use a bandit for allocation and reserve a small fixed-allocation slice for clean measurement.

Red flags (what NOT to say):

- "Bandits are strictly better because they waste less traffic"—ignores inference, guardrails, and delayed rewards.
- Describing Thompson sampling as "random exploration" with no posterior.
- Not knowing that adaptively collected data invalidates standard confidence intervals.

What the interviewer is testing: Whether you understand that bandits and A/B tests optimize different objectives—regret versus inference—rather than being faster and slower versions of the same thing.

What separates good from great:

- L5 Describes explore/exploit and names the algorithms.
- L6 Gives correct mechanics including Beta-Bernoulli updates, and a decision rule grounded in the objective.
- L7 Adds contextual bandits, propensity logging for off-policy evaluation, and non-stationarity handling.

Common follow-ups: "How would you evaluate a new policy offline from bandit logs?"

"What breaks under weekly seasonality?"

Interview Question

Q: We cannot randomize prices. How would you estimate price elasticity from historical data?

L7 Architecture Design ★★○

Strong answer:

Lead with the confounding story, because it is the whole problem: prices were not set randomly—they were raised when demand was strong and cut when inventory was stale—so a naive regression of quantity on price recovers the pricing policy, not the demand curve, and frequently returns an upward-sloping "demand." Controlling for observed covariates helps only if the pricing rule used nothing you cannot observe, which is rarely true.

Then map data shapes to designs. If prices follow a rule with a threshold (tier boundaries, an algorithmic cutoff), that is regression discontinuity—compare just above and below, check the density for manipulation, report bandwidth sensitivity. If price changes rolled out across markets at different times, that is difference-in-differences, and I would show the pre-trend/event-study plot before trusting it, being alert to the rollout order being correlated with market growth. If there is a supply-side cost shock or exchange-rate movement that shifts price without otherwise touching demand, that is an instrument—state the exclusion restriction and report the first-stage F. Panel data with market and time fixed effects absorbs a lot of the confounding but not time-varying market-level demand shocks, which is exactly the omitted variable here.

Close by pushing for an experiment, because that is the staff-level move: even a small randomized price test in a few markets, or a randomized promotion/discount as an encouragement design, identifies elasticity far more credibly than any of the above, and it is usually more feasible than people assume once the fairness and legal concerns are addressed (uniform published prices with randomized discounts, or geography-level randomization with disclosure). Propose the observational estimate as the interim answer with its assumption stated, and the experiment as the thing that settles it.

Red flags (what NOT to say):

- Regressing quantity on price with controls and calling it elasticity.
- Naming IV or DiD without stating the assumption each requires.
- Never questioning whether an experiment is truly impossible.
- Claiming propensity scores handle unobserved demand shocks.

What the interviewer is testing: Whether you reason from data-generating process to identification strategy, and whether you retain the instinct that a small experiment beats a clever observational estimate.

What separates good from great:

- L5 Recognizes confounding, proposes controls.
- L6 Matches a specific quasi-experimental design to the data shape with its assumption and diagnostic.
- L7 Sequences it: interim observational estimate with caveats, a targeted price experiment to validate, and a plan for continuous elasticity measurement via ongoing randomized variation.

Common follow-ups: "What is the LATE here and who are the compliers?" "How would you detect a violation of parallel trends?"

Interview Question

Q: What assumption does difference-in-differences require, and how would you probe it?

L5 Conceptual ★★○

Strong answer:

Parallel trends: absent the treatment, treated and control groups would have followed the same trajectory. Note precisely what it does *not* require—the groups may differ in level by any amount, since the first difference removes fixed group differences and the second removes shocks common to both.

Probe it by plotting the outcome for both groups over several pre-periods and running an event-study specification with leads and lags: coefficients on the leads should be statistically and visually flat. Supporting checks: a placebo test on a fake treatment date, a placebo outcome that treatment should not affect, and sensitivity to the choice of control group. State the honest caveat that parallel pre-trends are evidence, not proof—the assumption concerns the unobserved counterfactual path, and a differential shock coinciding with treatment breaks it regardless of clean pre-trends. Give a concrete violation: the feature launched in markets that were already accelerating, or a competitor exited one market at the same time. Composition change also breaks it—if treatment changes who is in the sample, the two periods measure different populations.

Red flags (what NOT to say):

- Saying the groups must be "similar" or "balanced" rather than parallel in trend.
- Treating matching pre-period *levels* as the assumption.
- Claiming a pre-trend check proves the assumption.

What the interviewer is testing: Whether you can state an identifying assumption precisely and name a diagnostic that could falsify it—the core competency for all observational work.

What separates good from great:

- L5 States parallel trends and suggests plotting pre-periods.
- L6 Adds event-study leads/lags, placebo tests, and a concrete violation story.
- L7 Notes staggered-adoption problems with two-way fixed effects and when to reach for modern estimators.

Common follow-ups: "What if only one treated unit exists?" (synthetic control). "How does staggered rollout complicate this?"

Interview Question

Q: Design the experimentation platform for a 200-engineer organization.

L7 Architecture Design ★★○

Strong answer:

Scope it as making correct experiments the path of least resistance, since at 200 engineers

the bottleneck is not statistics but the number of teams able to run a trustworthy test unaided.
Assignment and exposure: deterministic hashing of unit ID plus experiment salt for reproducible, stateless bucketing; a layered design (orthogonal domains) so many experiments run concurrently without colliding, with mutual-exclusion groups for changes that interact; exposure logged at the moment of *trigger*, not at page load, because trigger-based analysis is what makes small features detectable. Metrics: a central definition repository so "revenue" means one thing, with each experiment declaring one OEC plus a standard guardrail set that is evaluated automatically; per-metric variance and pre-period correlation precomputed so the platform can offer CUPED by default and quote an MDE at design time.
Analysis: automatic SRM checks that block a readout, delta-method or bootstrap standard errors for ratio metrics with clustering at the randomization unit, CUPED applied where ρ justifies it, Benjamini–Hochberg across exploratory metrics with the OEC and guardrails held at FWER, and pre-registered segments only. Fix a monitoring policy—fixed horizon by default, always-valid inference for teams that must act early, guardrail alerting always on—and enforce it in the tool rather than in documentation.
Trust and process: continuous A/A tests validating the pipeline's false-positive rate, an experiment registry that records hypothesis, MDE, duration, and outcome including failures, a lightweight design review for high-stakes tests and results review before launch, and long-run holdbacks to measure accumulated effects. Then the organizational half: a small central team owning the platform and methodology plus embedded analysts, self-serve documentation with the three or four rules that matter, and meta-analysis over the archive to calibrate how much to believe the next win. Success metrics for the platform itself: experiments per quarter, share with adequate power, share invalidated by SRM, and the gap between predicted and holdback-realized impact.

Red flags (what NOT to say):

- Only describing bucketing infrastructure, with no metric governance or trust layer.
- No SRM/A/A validation—the platform is then untrustworthy by construction.
- Letting every team define its own metrics and stopping rules.
- Ignoring the human process: reviews, registry, education.

What the interviewer is testing: Whether you think in systems and incentives rather than statistics alone—the platform's job is to make the statistically correct action the default one.

What separates good from great:

- L5 Describes assignment and a results dashboard.
- L6 Adds metric governance, CUPED, SRM gating, and multiplicity policy.
- L7 Adds trust infrastructure (A/A, registry, holdbacks), an explicit monitoring policy, and platform-level success metrics.

Common follow-ups: "How would you handle interference at this scale?" "What would you centralize versus federate?"

Chapter 7

Time Series Essentials

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).

7.1 Foundations and Stationarity

7.2 Classical Models

7.3 The GBM Recipe

7.4 When Deep Learning

7.5 Interview Questions

Chapter 8

The From-Scratch Coding Canon

TL;DR

Placeholder — this chapter is written per docs/coverage-spec.md, seeded by the author's own conventional-ML document when it arrives (Phase C3).